PROPUESTA DE PROYECTO DE SOFTWARE

Licorería Junín Currinchero

Versión académica organizada a partir de la propuesta original y la Actividad Formativa N.º 1. Arquitectura seleccionada: arquitectura por capas.

Tecnologías: Ionic, Angular, TypeScript, JavaScript, Capacitor, Node.js, Express.js, Sequelize y MySQL. Plataformas: móvil y web para computadoras.

1. Tema del proyecto

Desarrollo de una aplicación multiplataforma (móvil y web) para la gestión de ventas, inventario y compras de clientes de la Licorería Junín Currinchero mediante Ionic, Angular y Node.js.

2. Descripción, problemática y propósito

Ampliación funcional propuesta: los asistentes AAC-001, AAC-002 y AAC-003 permitirán orientar la búsqueda de productos, consultar el seguimiento de pedidos propios y responder preguntas frecuentes, respectivamente. Sus respuestas se fundamentarán en datos autorizados y estarán sujetas a revisión, sin sustituir la gestión administrativa.

La Licorería Junín Currinchero requiere organizar el control de productos, existencias, ventas y pedidos. La gestión manual puede dificultar la actualización del inventario y la atención de solicitudes. Se propone una aplicación multiplataforma, accesible desde dispositivos móviles y navegadores de computadoras, con área de usuarios y área administrativa, conectada a una API REST y a una base de datos MySQL.

Su propósito es mejorar la administración comercial, facilitar la consulta de productos y la creación de pedidos, agilizar la atención y conservar información consistente de ventas y existencias.

2.1. Asistencia inteligente propuesta

Se propone incorporar asistencia inteligente para consultar productos, realizar seguimiento de pedidos propios y orientar en preguntas frecuentes. Los datos se obtendrán mediante servicios autorizados del backend; las respuestas no sustituirán la verificación administrativa de pagos ni la atención humana.

3. Alcance y limitaciones

Alcance de IA: la asistencia se incorporará a las vistas actuales de catálogo, seguimiento y ayuda; no creará un tercer rol ni nuevas vistas principales. Los agentes no podrán aprobar transferencias, alterar estados de pedidos ni acceder directamente a MySQL. El proveedor, los costos y la conservación de consultas quedan pendientes de decisión.

El alcance previsto comprende una aplicación multiplataforma para Android y navegadores web en computadoras, con posibilidad futura de adaptación a iOS, para clientes adultos y personal autorizado. Incluye autenticación, catálogo, carrito, pedidos, seguimiento, pagos en efectivo o transferencia mediante banca móvil, gestión de productos, categorías, inventario, ventas, clientes, usuarios, reportes y configuración.

Límites y decisiones pendientes: el documento no establece integración automática con bancos, pasarelas de pago, logística de terceros ni operación sin conexión. La transferencia por banca móvil se comprueba administrativamente; el comprobante por sí solo no acredita recepción. Se

deberán concretar zonas de entrega, horarios y verificación efectiva de mayoría de edad antes de la implementación.

3.1. Alcance y restricciones de los agentes inteligentes

AAC-001, AAC-002 y AAC-003 se integrarán en vistas existentes, sin aumentar las 21 vistas principales. No tendrán acceso directo a MySQL, no efectuarán compras ni cambiarán estados de pedidos o pagos. Los resultados deberán identificar falta de datos y permitir derivación a personal autorizado. Su modelo, proveedor, costos y tratamiento de datos quedan pendientes de aprobación.

4. Módulos, vistas y funcionalidades

Integración en vistas existentes: Catálogo y búsqueda → AAC-001 (productos, precios y existencias vigentes); Seguimiento → AAC-002 (solo pedidos del usuario autenticado); Ayuda y contacto → AAC-003 (horarios, pagos, entregas y derivación humana). Se conservan las 21 vistas principales y los roles Administrador y Usuario.

Se distinguen las 21 vistas principales del prototipo de los módulos lógicos. Autenticación es transversal; búsqueda y filtros es una subfuncionalidad del catálogo; detalle y actualización de pedido es una subvista de gestión de pedidos. El menú lateral es un componente de navegación, no un módulo de negocio.

Área	Módulo / vista	Funcionalidades principales
Compartido	Registro e inicio de sesión	Registro de clientes, inicio y cierre de sesión, recuperación de contraseña, roles, permisos y verificación de edad.
Cliente	Inicio	Accesos al catálogo, carrito, pedidos, perfil, categorías y destacados.
Cliente	Catálogo y búsqueda	Productos activos, detalle, precios, disponibilidad, búsqueda, filtros y ordenamiento.
Cliente	Carrito	Agregar, cambiar cantidades, quitar productos, subtotal y validación de stock.
Cliente	Confirmación de pedidos	Modalidad de entrega, efectivo o banca móvil, comprobante, total y registro del pedido.
Cliente	Seguimiento	Estado y progreso, detalle, fechas, cancelaciones permitidas y avisos.
Cliente	Historial de compras	Compras anteriores, detalle, importes, fechas y estados.
Cliente	Perfil	Datos personales, direcciones, contraseña y preferencias.
Cliente	Notificaciones	Avisos de pedidos y pagos, consulta y marcado de lectura.
Cliente	Ayuda y contacto	Preguntas frecuentes, canales de contacto y solicitudes de soporte
Administrador	Dashboard	Indicadores de ventas, pedidos, productos, alertas y gráficos.
Administrador	Productos	Crear, consultar, editar, activar, desactivar y retirar productos.
Administrador	Inventario	Entradas, salidas, ajustes, alertas e historial de movimientos.

Administrador	Pedidos	Consultar, confirmar, rechazar, actualizar estados y verificar pagos; incluye detalle de pedido.
Administrador	Ventas	Registrar ventas presenciales, calcular totales y vincular pedidos sin duplicados
Administrador	Categorías	Crear, editar, consultar, activar y desactivar categorías.
Administrador	Clientes	Consultar perfiles, historial y estado de cuentas.
Administrador	Usuarios y permisos	Crear cuentas autorizadas, asignar roles y auditar cambios.
Administrador	Reportes	Ventas, productos, inventario, pedidos y exportaciones.
Administrador	Configuración	Horarios, contactos, entregas, medios de pago, políticas y stock mínimo.
Administrador	Atención al cliente	Recibir, responder, asignar y cerrar solicitudes de soporte.

5. Requerimientos funcionales (RF)

Ampliación: los requisitos RF-241 a RF-255, con sus criterios de aceptación y pruebas propuestas, se presentan en esta misma sección, después de los requisitos originales. No modifican los RF-001 a RF-240.

Se conservan los identificadores RF-001 a RF-240 y la redacción de base del documento original. Los RF-001 a RF-014 corresponden a autenticación; RF-015 a RF-108 al cliente; RF-109 a RF-226 al administrador; RF-227 a RF-240 a procesos transversales.

Autenticación compartida

Código	Módulo	Requerimiento funcional
RF-001	Autenticación compartida	El sistema deberá permitir registrar nuevas cuentas de clientes.
RF-002	Autenticación compartida	El sistema deberá solicitar nombre completo, correo electrónico, teléfono y contraseña.
RF-003	Autenticación compartida	El sistema deberá validar los campos obligatorios y el formato de los datos.
RF-004	Autenticación compartida	El sistema deberá impedir el registro de correos electrónicos duplicados.
RF-005	Autenticación compartida	El sistema deberá solicitar la aceptación de términos y condiciones.
RF-006	Autenticación compartida	El sistema deberá implementar un mecanismo de verificación de mayoría de edad antes de habilitar compras.
RF-007	Autenticación compartida	El sistema deberá permitir iniciar sesión mediante correo electrónico y contraseña.
RF-008	Autenticación compartida	El sistema deberá identificar el rol del usuario autenticado.

Código	Módulo	Requerimiento funcional
RF-009	Autenticación compartida	El sistema deberá dirigir al cliente a su pantalla de inicio y al administrador al Dashboard.
RF-010	Autenticación compartida	El sistema deberá permitir recuperar y restablecer contraseñas.
RF-011	Autenticación compartida	El sistema deberá permitir cerrar sesión.
RF-012	Autenticación compartida	El sistema deberá impedir el acceso a cuentas desactivadas o bloqueadas.
RF-013	Autenticación compartida	El sistema deberá restringir el acceso a los módulos según el rol y los permisos asignados.
RF-014	Autenticación compartida	El sistema deberá impedir el registro público de cuentas administrativas.
RF-015	Cliente	Mostrar la pantalla principal del cliente autenticado.
RF-016	Cliente	Mostrar accesos al catálogo, carrito, pedidos y perfil.
RF-017	Cliente	Mostrar productos destacados configurados por el administrador.
RF-018	Cliente	Mostrar las categorías disponibles.
RF-019	Cliente	Mostrar información general y horarios del negocio.
RF-020	Cliente	Permitir navegar a los módulos autorizados del cliente.
RF-021	Cliente	Mostrar los productos activos registrados.
RF-022	Cliente	Mostrar la fotografía de cada producto.
RF-023	Cliente	Mostrar nombre, descripción, precio y categoría.
RF-024	Cliente	Mostrar disponibilidad del producto.
RF-025	Cliente	Permitir consultar el detalle de un producto.
RF-026	Cliente	Mostrar las presentaciones o tamaños disponibles, cuando corresponda.
RF-027	Cliente	Mostrar los precios actualizados por el administrador.
RF-028	Cliente	Impedir agregar al carrito productos agotados o inactivos.
RF-029	Cliente	Permitir agregar productos disponibles al carrito desde el catálogo o su detalle.
RF-030	Cliente	Buscar productos por nombre.
RF-031	Cliente	Buscar productos mediante palabras clave.
RF-032	Cliente	Filtrar productos por categoría.
RF-033	Cliente	Filtrar productos por disponibilidad.
RF-034	Cliente	Ordenar productos de menor a mayor precio.
RF-035	Cliente	Ordenar productos de mayor a menor precio.
RF-036	Cliente	Combinar búsqueda, filtros y ordenamiento.
RF-037	Cliente	Mostrar un mensaje cuando no existan resultados.
RF-038	Cliente	Agregar productos al carrito.

Código	Módulo	Requerimiento funcional
RF-039	Cliente	Mostrar productos agregados.
RF-040	Cliente	Incrementar la cantidad de un producto.
RF-041	Cliente	Disminuir la cantidad de un producto.
RF-042	Cliente	Eliminar productos del carrito.
RF-043	Cliente	Vaciar completamente el carrito.
RF-044	Cliente	Calcular automáticamente el subtotal.
RF-045	Cliente	Mostrar el número total de unidades seleccionadas.
RF-046	Cliente	Validar las existencias disponibles antes de confirmar la compra.
RF-047	Cliente	Conservar temporalmente el carrito durante la navegación y según la política de persistencia definida.
RF-048	Cliente	Mostrar el resumen de productos, cantidades y precios.
RF-049	Cliente	Solicitar y validar un teléfono de contacto.
RF-050	Cliente	Permitir seleccionar retiro en el establecimiento.
RF-051	Cliente	Permitir seleccionar entrega a domicilio cuando esté habilitada.
RF-052	Cliente	Solicitar la dirección cuando se seleccione entrega a domicilio.
RF-053	Cliente	Permitir seleccionar pago en efectivo.
RF-054	Cliente	Permitir seleccionar transferencia mediante banca móvil.
RF-055	Cliente	Mostrar los datos de la cuenta receptora configurada por el negocio.
RF-056	Cliente	Mostrar el importe exacto que debe transferirse.
RF-057	Cliente	Permitir ingresar el número de referencia de la transferencia.
RF-058	Cliente	Permitir adjuntar un comprobante de pago.
RF-059	Cliente	Registrar el pago por banca móvil como pendiente de verificación.
RF-060	Cliente	Permitir ingresar observaciones adicionales.
RF-061	Cliente	Calcular subtotal, costo de entrega y total.
RF-062	Cliente	Validar las condiciones de compra, disponibilidad y verificación de edad requerida.
RF-063	Cliente	Registrar el pedido y generar un identificador único.
RF-064	Cliente	Mostrar la confirmación y el estado inicial del pedido.
RF-065	Cliente	Mostrar los pedidos realizados por el cliente autenticado.
RF-066	Cliente	Consultar el detalle de un pedido.
RF-067	Cliente	Mostrar el estado actual del pedido.
RF-068	Cliente	Mostrar estados como pendiente, confirmado, en preparación, listo, entregado o cancelado, según corresponda.
RF-069	Cliente	Mostrar los cambios de estado registrados.
RF-070	Cliente	Mostrar productos, cantidades y total.
RF-071	Cliente	Mostrar el método y estado del pago.

Código	Módulo	Requerimiento funcional
RF-072	Cliente	Permitir solicitar la cancelación cuando las condiciones lo permitan.
RF-073	Cliente	Notificar al cliente cuando cambie el estado de su pedido.
RF-074	Cliente	Consultar los pedidos anteriores del cliente.
RF-075	Cliente	Mostrar el número y fecha de cada pedido.
RF-076	Cliente	Mostrar los productos y cantidades compradas.
RF-077	Cliente	Mostrar el total pagado o pendiente.
RF-078	Cliente	Consultar el estado final del pedido.
RF-079	Cliente	Filtrar pedidos por fecha o estado.
RF-080	Cliente	Consultar el detalle histórico de una compra.
RF-081	Cliente	Mostrar la información personal del cliente.
RF-082	Cliente	Permitir actualizar el nombre.
RF-083	Cliente	Permitir actualizar el teléfono.
RF-084	Cliente	Permitir actualizar el correo mediante verificación cuando corresponda.
RF-085	Cliente	Permitir registrar direcciones de entrega.
RF-086	Cliente	Permitir editar o eliminar direcciones.
RF-087	Cliente	Permitir cambiar la contraseña.
RF-088	Cliente	Validar los cambios antes de guardarlos.
RF-089	Cliente	Permitir cerrar sesión desde el perfil.
RF-090	Cliente	Mostrar notificaciones relacionadas con los pedidos.
RF-091	Cliente	Notificar cuando un pedido sea confirmado.
RF-092	Cliente	Notificar cambios de estado del pedido.
RF-093	Cliente	Notificar cuando un pedido esté listo para retirar o entregar.
RF-094	Cliente	Notificar el resultado de la verificación del pago.
RF-095	Cliente	Permitir marcar notificaciones como leídas.
RF-096	Cliente	Permitir consultar el historial de notificaciones.
RF-097	Cliente	Permitir eliminar u ocultar notificaciones de la bandeja del usuario.
RF-098	Cliente	Mostrar un centro de ayuda.
RF-099	Cliente	Consultar preguntas frecuentes.
RF-100	Cliente	Buscar preguntas frecuentes por palabras clave.
RF-101	Cliente	Mostrar teléfono, correo y medios oficiales de contacto.
RF-102	Cliente	Permitir abrir el canal oficial de WhatsApp.
RF-103	Cliente	Mostrar ubicación y horarios de atención.
RF-104	Cliente	Mostrar políticas de pedidos, pagos, retiro y entrega.
RF-105	Cliente	Permitir enviar solicitudes de soporte.

Código	Módulo	Requerimiento funcional
RF-106	Cliente	Permitir reportar problemas relacionados con pedidos.
RF-107	Cliente	Permitir consultar el estado de una solicitud.
RF-108	Cliente	Mostrar las respuestas y el historial de solicitudes del cliente.
RF-109	Administrador	Mostrar un resumen general del negocio.
RF-110	Administrador	Mostrar el total de ventas del período seleccionado.
RF-111	Administrador	Mostrar el número de productos registrados.
RF-112	Administrador	Mostrar el número de pedidos recibidos.
RF-113	Administrador	Mostrar los pedidos pendientes.
RF-114	Administrador	Mostrar productos con existencias bajas.
RF-115	Administrador	Mostrar indicadores y gráficos de ventas.
RF-116	Administrador	Mostrar pedidos recientes.
RF-117	Administrador	Permitir actualizar los indicadores del panel.
RF-118	Administrador	Registrar nuevos productos.
RF-119	Administrador	Editar la información de productos existentes.
RF-120	Administrador	Registrar nombre y descripción.
RF-121	Administrador	Asignar precio de venta.
RF-122	Administrador	Adjuntar o actualizar fotografías.
RF-123	Administrador	Asignar una categoría.
RF-124	Administrador	Consultar el listado de productos.
RF-125	Administrador	Buscar productos por nombre.
RF-126	Administrador	Filtrar productos por categoría o estado.
RF-127	Administrador	Activar o desactivar productos.
RF-128	Administrador	Eliminar productos cuando las reglas de integridad lo permitan o desactivarlos cuando tengan historial asociado.
RF-129	Administrador	Validar que los precios y datos registrados sean correctos.
RF-130	Administrador	Consultar las existencias actuales.
RF-131	Administrador	Registrar entradas de inventario.
RF-132	Administrador	Registrar salidas de inventario.
RF-133	Administrador	Registrar ajustes de existencias.
RF-134	Administrador	Registrar el motivo de cada ajuste.
RF-135	Administrador	Definir niveles mínimos de stock.
RF-136	Administrador	Detectar productos con stock bajo.
RF-137	Administrador	Mostrar productos agotados.
RF-138	Administrador	Consultar el historial de movimientos.
RF-139	Administrador	Impedir operaciones que generen existencias negativas.

Código	Módulo	Requerimiento funcional
RF-140	Administrador	Actualizar el inventario cuando se registren ventas o pedidos, según las reglas definidas.
RF-141	Administrador	Consultar todos los pedidos recibidos.
RF-142	Administrador	Filtrar pedidos por estado.
RF-143	Administrador	Buscar pedidos por número o cliente.
RF-144	Administrador	Consultar el detalle de cada pedido.
RF-145	Administrador	Mostrar productos, cantidades y precios.
RF-146	Administrador	Consultar datos de contacto y entrega.
RF-147	Administrador	Confirmar pedidos válidos.
RF-148	Administrador	Rechazar pedidos e indicar el motivo.
RF-149	Administrador	Actualizar el estado de preparación.
RF-150	Administrador	Marcar pedidos como listos para retirar o entregar.
RF-151	Administrador	Marcar pedidos como entregados.
RF-152	Administrador	Gestionar cancelaciones permitidas.
RF-153	Administrador	Consultar el comprobante de banca móvil.
RF-154	Administrador	Verificar la recepción del pago y aprobarlo o rechazarlo.
RF-155	Administrador	Registrar y notificar los cambios de estado del pedido y del pago.
RF-156	Administrador	Registrar ventas realizadas en el establecimiento.
RF-157	Administrador	Seleccionar productos para una venta.
RF-158	Administrador	Registrar cantidades vendidas.
RF-159	Administrador	Consultar el precio unitario.
RF-160	Administrador	Calcular subtotales automáticamente.
RF-161	Administrador	Calcular el total de la venta.
RF-162	Administrador	Registrar el método de pago.
RF-163	Administrador	Confirmar la venta.
RF-164	Administrador	Actualizar las existencias correspondientes.
RF-165	Administrador	Consultar ventas registradas por fecha.
RF-166	Administrador	Asociar la venta a un pedido cuando corresponda, evitando duplicar el ingreso.
RF-167	Administrador	Registrar nuevas categorías.
RF-168	Administrador	Editar nombres de categorías.
RF-169	Administrador	Registrar descripciones de categorías.
RF-170	Administrador	Consultar las categorías existentes.
RF-171	Administrador	Mostrar el número de productos por categoría.
RF-172	Administrador	Activar o desactivar categorías.

Código	Módulo	Requerimiento funcional
RF-173	Administrador	Eliminar categorías sin relaciones que impidan su eliminación.
RF-174	Administrador	Evitar categorías duplicadas.
RF-175	Administrador	Consultar clientes registrados.
RF-176	Administrador	Buscar clientes por nombre o correo.
RF-177	Administrador	Consultar datos de contacto.
RF-178	Administrador	Consultar el detalle del cliente.
RF-179	Administrador	Consultar su historial de pedidos.
RF-180	Administrador	Consultar compras realizadas y sus totales.
RF-181	Administrador	Actualizar datos administrativos permitidos.
RF-182	Administrador	Activar o desactivar cuentas según permisos y procedimientos autorizados.
RF-183	Administrador	Restringir la consulta de datos personales a personal autorizado.
RF-184	Administrador	Crear cuentas administrativas mediante un procedimiento autorizado.
RF-185	Administrador	Consultar las cuentas del sistema según los permisos.
RF-186	Administrador	Asignar roles a usuarios autorizados.
RF-187	Administrador	Configurar permisos por módulo o acción.
RF-188	Administrador	Modificar permisos existentes.
RF-189	Administrador	Activar cuentas.
RF-190	Administrador	Desactivar cuentas.
RF-191	Administrador	Restablecer el acceso mediante un procedimiento seguro.
RF-192	Administrador	Impedir que usuarios sin autorización accedan a funciones administrativas.
RF-193	Administrador	Registrar cambios relevantes de roles, permisos y estados de cuentas.
RF-194	Administrador	Generar reportes de ventas.
RF-195	Administrador	Filtrar reportes por fechas.
RF-196	Administrador	Mostrar ventas diarias, semanales y mensuales.
RF-197	Administrador	Calcular ingresos por período.
RF-198	Administrador	Mostrar productos más vendidos.
RF-199	Administrador	Mostrar productos con menor movimiento.
RF-200	Administrador	Generar reportes de inventario.
RF-201	Administrador	Mostrar productos con stock bajo.
RF-202	Administrador	Generar reportes de pedidos por estado.
RF-203	Administrador	Mostrar indicadores mediante gráficos.
RF-204	Administrador	Exportar reportes en PDF o CSV.
RF-205	Administrador	Permitir consultar el detalle de los datos que sustentan cada reporte.

Código	Módulo	Requerimiento funcional
RF-206	Administrador	Registrar y actualizar el nombre comercial.
RF-207	Administrador	Registrar dirección y ubicación.
RF-208	Administrador	Configurar teléfonos y canales de contacto.
RF-209	Administrador	Configurar horarios de atención.
RF-210	Administrador	Habilitar o deshabilitar entrega a domicilio.
RF-211	Administrador	Configurar condiciones de retiro y entrega.
RF-212	Administrador	Configurar costos de entrega.
RF-213	Administrador	Habilitar o deshabilitar métodos de pago.
RF-214	Administrador	Configurar los datos receptores para transferencias mediante banca móvil.
RF-215	Administrador	Configurar políticas de compra, cancelación y devolución aplicables.
RF-216	Administrador	Configurar valores predeterminados de stock mínimo.
RF-217	Administrador	Guardar y aplicar los cambios autorizados.
RF-218	Administrador	Consultar las solicitudes de soporte recibidas.
RF-219	Administrador	Filtrar solicitudes por estado.
RF-220	Administrador	Consultar el detalle de una solicitud.
RF-221	Administrador	Consultar el cliente y pedido relacionado, cuando corresponda.
RF-222	Administrador	Asignar solicitudes a personal autorizado.
RF-223	Administrador	Responder consultas de clientes.
RF-224	Administrador	Cambiar estados: pendiente, en atención y resuelta.
RF-225	Administrador	Consultar el historial de respuestas.
RF-226	Administrador	Notificar al cliente cuando exista una respuesta o resolución.
RF-227	Transversales	Centralizar la autenticación de clientes y administradores.
RF-228	Transversales	Aplicar autorización por roles y permisos en el backend.
RF-229	Transversales	Sincronizar la información del catálogo con la gestión administrativa.
RF-230	Transversales	Mantener la consistencia entre pedidos, ventas e inventario.
RF-231	Transversales	Registrar operaciones críticas mediante transacciones de base de datos.
RF-232	Transversales	Evitar confirmar pedidos cuando las existencias sean insuficientes.
RF-233	Transversales	Evitar registrar dos veces una misma venta o confirmación de pago.
RF-234	Transversales	Validar los datos tanto en Angular como en Node.js.
RF-235	Transversales	Registrar auditorías de operaciones administrativas relevantes.
RF-236	Transversales	Mostrar mensajes claros de éxito, error y validación.
RF-237	Transversales	Mantener la integridad de las relaciones entre usuarios, productos, pedidos y ventas.
RF-238	Transversales	Controlar el acceso a los comprobantes de pago.

Código	Módulo	Requerimiento funcional
RF-239	Transversales	Mantener el historial de estados de pedidos y pagos.
RF-240	Transversales	Aplicar la verificación de edad y las restricciones legales correspondientes antes de completar operaciones de compra y entrega de alcohol.

5.1. Requisitos funcionales adicionales (RF-241 a RF-255)

Código	Módulo	Requerimiento funcional	Aceptación verificable / prueba propuesta
RF-241	Consulta de productos	Mostrar un asistente dentro del catálogo que reciba preguntas sobre productos.	Una consulta válida se envía al servicio autorizado y se presenta su respuesta. Prueba: Abrir asistente y enviar pregunta.
RF-242	Consulta de productos	Consultar precio, categoría y existencias mediante API sin inventar valores.	La respuesta coincide con el dato devuelto por la API; si falta, informa que no está disponible. Prueba: Comparar respuestas con catálogo de prueba.
RF-243	Consulta de productos	Permitir localizar productos por nombre, categoría o características registradas.	Los productos sugeridos existen y están autorizados para mostrarse. Prueba: Buscar artículos existentes e inexistentes.
RF-244	Seguimiento	Permitir preguntar por el estado de un pedido del usuario autenticado.	Se devuelve solo un pedido cuyo propietario coincide con el usuario de la sesión. Prueba: Consultar pedido propio.
RF-245	Seguimiento	Rechazar acceso a pedidos ajenos o solicitudes sin sesión válida.	El backend responde con rechazo y no revela información del pedido. Prueba: Intentar ID ajeno y sesión inválida.
RF-246	Seguimiento	Mostrar estado y fecha de actualización registrados sin modificar pedidos.	Los datos coinciden con la API; no se ejecutan operaciones de escritura. Prueba: Comparar estados y revisar auditoría.
RF-247	Soporte	Responder preguntas frecuentes desde contenido aprobado del negocio.	Respuesta basada en una fuente vigente o mensaje de información no disponible. Prueba: Preguntar horarios y políticas.
RF-248	Soporte	Orientar sobre pago en efectivo y transferencia mediante banca móvil.	Indica que el comprobante no equivale a pago verificado. Prueba: Preguntar por confirmación de transferencia.
RF-249	Soporte	Derivar consultas no resueltas al canal de atención humana.	Ofrece el canal oficial o registra solicitud autorizada. Prueba: Simular pregunta fuera de alcance.

RF-250	Transversal	Identificar claramente que la respuesta proviene de un asistente de IA.	La interfaz presenta identificación visible del asistente. Prueba: Inspección de vista.
RF-251	Transversal	Aplicar autenticación, autorización y minimización de datos en los endpoints de IA.	Las pruebas de permisos bloquean accesos indebidos. Prueba: Pruebas de rol y datos sensibles.
RF-252	Transversal	Registrar eventos técnicos y errores de agentes sin almacenar secretos.	La auditoría registra identificador, resultado y fecha sin tokens ni contraseñas. Prueba: Revisar logs de prueba.
RF-253	Transversal	Permitir reintento seguro y respuesta de contingencia cuando el servicio de IA falle.	No hay duplicación de acciones; se informa indisponibilidad. Prueba: Simular fallo y tiempo de espera.
RF-254	Transversal	Proteger las instrucciones internas frente a solicitudes de modificación de reglas.	La consulta maliciosa no altera permisos ni expone instrucciones privadas. Prueba: Ensayar prompt injection.
RF-255	Transversal	Solicitar confirmación humana para cualquier acción administrativa o financiera no permitida al agente.	No existe endpoint de agente para confirmar pagos o cambiar estados. Prueba: Revisar rutas y probar solicitudes.

6. Requerimientos no funcionales (RNF)

Ampliación: los requisitos RNF-061 a RNF-072 complementan seguridad, privacidad, exactitud, accesibilidad y desempeño de la asistencia inteligente. Se conservan íntegros RNF-001 a RNF-060.

Se mantienen los 60 RNF y sus metas de aceptación como objetivos propuestos, no como resultados ya medidos. Para preservar las formulaciones originales, se transcribe la sección de RNF del documento base por categorías.

Los requerimientos no funcionales establecen las características de calidad, seguridad, rendimiento, compatibilidad y mantenimiento que debe cumplir la aplicación.

Los valores numéricos siguientes son metas propuestas para el proyecto, que deberán verificarse durante las pruebas; no representan resultados ya medidos.

Código	Categoría	Requerimiento no funcional	Criterio de aceptación propuesto
RNF-001	Rendimiento y eficiencia	La aplicación deberá iniciar rápidamente.	Pantalla inicial visible en máximo 3 segundos en un dispositivo de prueba definido.
RNF-002	Rendimiento y eficiencia	Las pantallas principales deberán cargar con fluidez.	Máximo 3 segundos en condiciones normales de red.
RNF-003	Rendimiento y eficiencia	Las consultas habituales a la API deberán responder rápidamente.	Percentil 95 de respuesta inferior a 1 segundo, sin contar la transferencia de archivos.

Código	Categoría	Requerimiento no funcional	Criterio de aceptación propuesto
RNF-004	Rendimiento y eficiencia	El sistema deberá admitir usuarios simultáneos.	Soportar al menos 50 sesiones concurrentes en una prueba de carga.
RNF-005	Rendimiento y eficiencia	Las búsquedas deberán ser eficientes.	Resultados en máximo 2 segundos para un catálogo de prueba de 1.000 productos.
RNF-006	Rendimiento y eficiencia	La interfaz deberá evitar bloqueos innecesarios.	Mostrar indicadores de carga durante operaciones prolongadas.
RNF-007	Rendimiento y eficiencia	La carga de imágenes deberá optimizarse.	Usar imágenes comprimidas y carga diferida donde corresponda.
RNF-008	Seguridad	Las contraseñas deberán almacenarse de manera segura.	Utilizar Argon2id o bcrypt con parámetros adecuados; nunca texto plano.
RNF-009	Seguridad	La comunicación deberá ser cifrada.	Utilizar HTTPS/TLS en producción.
RNF-010	Seguridad	La autenticación deberá ser segura.	Usar JWT con expiración y validación de firma.
RNF-011	Seguridad	Las credenciales de sesión deberán protegerse.	Utilizar almacenamiento seguro apropiado para la plataforma móvil.
RNF-012	Seguridad	Los permisos deberán validarse en el servidor.	Una petición sin permiso deberá ser rechazada, aunque se modifique la interfaz.
RNF-013	Seguridad	El sistema deberá prevenir inyección SQL.	Utilizar consultas parametrizadas mediante Sequelize y validar entradas.
RNF-014	Seguridad	Los intentos de acceso deberán controlarse.	Aplicar limitación de intentos y medidas contra ataques de fuerza bruta.
RNF-015	Seguridad	Los archivos cargados deberán validarse.	Comprobar tamaño, tipo real, extensión y permisos de acceso.
RNF-016	Seguridad	Los datos personales deberán protegerse.	Aplicar controles de acceso, minimización y políticas de conservación.
RNF-017	Seguridad	Los eventos administrativos críticos deberán ser auditables.	Registrar usuario, acción, fecha y resultado, sin incluir contraseñas ni secretos.
RNF-018	Seguridad	La aplicación deberá evitar exposición de información sensible.	No mostrar tokens, claves ni datos bancarios sensibles en registros de errores.
RNF-019	Usabilidad y accesibilidad	La interfaz deberá ser intuitiva.	Usuarios de prueba deberán completar tareas esenciales sin asistencia, según un guion de evaluación.
RNF-020	Usabilidad y accesibilidad	La navegación deberá ser consistente.	Mantener encabezados, botones y navegación coherentes entre módulos.
RNF-021	Usabilidad y accesibilidad	Los formularios deberán ser comprensibles.	Mostrar etiquetas y mensajes de validación junto a los campos.

Código	Categoría	Requerimiento no funcional	Criterio de aceptación propuesto
RNF-022	Usabilidad y accesibilidad	La interfaz deberá adaptarse a diferentes tamaños de pantalla.	No presentar desbordamientos horizontales en los dispositivos de prueba.
RNF-023	Usabilidad y accesibilidad	Los controles táctiles deberán ser accesibles.	Utilizar áreas táctiles de tamaño adecuado, como mínimo 44 × 44 puntos CSS equivalentes cuando corresponda.
RNF-024	Usabilidad y accesibilidad	El texto deberá ser legible.	Mantener contraste suficiente y permitir el escalado de texto.
RNF-025	Usabilidad y accesibilidad	El sistema deberá informar el resultado de las acciones.	Mostrar confirmaciones y errores comprensibles.
RNF-026	Usabilidad y accesibilidad	La aplicación deberá utilizar español.	Todas las vistas principales y mensajes estarán disponibles en español.
RNF-027	Disponibilidad y confiabilidad	El backend deberá mantener una disponibilidad adecuada.	Meta de 99 % mensual durante el período de operación evaluado.
RNF-028	Disponibilidad y confiabilidad	Los datos deberán respaldarse periódicamente.	Realizar copias de seguridad diarias de MySQL.
RNF-029	Disponibilidad y confiabilidad	Los respaldos deberán poder restaurarse.	Ejecutar pruebas de restauración periódicas.
RNF-030	Disponibilidad y confiabilidad	Los pedidos deberán conservar su integridad.	No permitir pedidos incompletos por fallos parciales de escritura.
RNF-031	Disponibilidad y confiabilidad	Los movimientos de inventario deberán ser consistentes.	Evitar stock negativo y actualizaciones simultáneas conflictivas.
RNF-032	Disponibilidad y confiabilidad	Los errores de conexión deberán manejarse adecuadamente.	Informar al usuario y permitir reintentos seguros.
RNF-033	Disponibilidad y confiabilidad	Las operaciones críticas deberán evitar duplicados.	Aplicar mecanismos de idempotencia en confirmaciones y registros de pago.
RNF-034	Compatibilidad y portabilidad	La aplicación deberá funcionar en Android.	Probar en versiones Android definidas como compatibles.
RNF-035	Compatibilidad y portabilidad	La solución deberá permitir una futura versión iOS.	Utilizar componentes Ionic y plugins compatibles con Capacitor, cuando sea viable.
RNF-036	Compatibilidad y portabilidad	La interfaz deberá ser adaptable.	Probar en teléfonos con distintas resoluciones.
RNF-037	Compatibilidad y portabilidad	El backend deberá ser independiente del sistema móvil.	Exponer funcionalidades mediante API REST.
RNF-038	Compatibilidad y portabilidad	Las herramientas deberán ser compatibles entre sí.	Documentar versiones de Node.js, Angular, Ionic, Capacitor y dependencias.
RNF-039	Mantenibilidad y arquitectura	El backend deberá utilizar arquitectura por capas.	Separar rutas, controladores, servicios y acceso a datos.
RNF-040	Mantenibilidad y arquitectura	El frontend deberá estar organizado por módulos funcionales.	Separar autenticación, cliente, administrador y componentes compartidos.

Código	Categoría	Requerimiento no funcional	Criterio de aceptación propuesto
RNF-041	Mantenibilidad y arquitectura	El código deberá ser legible y consistente.	Aplicar convenciones de nombres, formato y análisis estático.
RNF-042	Mantenibilidad y arquitectura	El proyecto deberá utilizar control de versiones.	Mantener el código en un repositorio Git.
RNF-043	Mantenibilidad y arquitectura	La aplicación deberá permitir incorporar módulos nuevos.	Evitar dependencias directas innecesarias entre módulos.
RNF-044	Mantenibilidad y arquitectura	La lógica de negocio deberá centralizarse.	Implementar reglas críticas en servicios del backend.
RNF-045	Mantenibilidad y arquitectura	El sistema deberá disponer de documentación técnica.	Documentar instalación, configuración, API y estructura del proyecto.
RNF-046	Mantenibilidad y arquitectura	El sistema deberá contar con pruebas automatizadas.	Probar autenticación, pedidos, inventario y operaciones críticas.
RNF-047	Base de datos e integridad	MySQL deberá mantener integridad referencial.	Definir claves primarias, foráneas y restricciones.
RNF-048	Base de datos e integridad	Las consultas frecuentes deberán estar optimizadas.	Crear índices para búsquedas de productos, usuarios y pedidos.
RNF-049	Base de datos e integridad	Las operaciones relacionadas deberán ser atómicas.	Utilizar transacciones para pedidos, ventas y movimientos de stock.
RNF-050	Base de datos e integridad	Los datos monetarios deberán manejarse con precisión.	Usar DECIMAL en MySQL y evitar cálculos monetarios con coma flotante sin control.
RNF-051	Base de datos e integridad	Los registros históricos deberán conservar consistencia.	Guardar precios y cantidades de cada operación al momento de su registro.
RNF-052	Privacidad, cumplimiento y operación	El tratamiento de datos deberá cumplir la normativa aplicable en Ecuador.	Documentar finalidades, base legal, acceso y conservación conforme a la LOPDP.
RNF-053	Privacidad, cumplimiento y operación	La compra de bebidas alcohólicas deberá respetar las restricciones legales.	Implementar verificación efectiva de mayoría de edad y controles de entrega.
RNF-054	Privacidad, cumplimiento y operación	La aplicación deberá respetar las condiciones legales de comercialización.	Configurar restricciones de horarios, entrega y venta según la normativa aplicable.
RNF-055	Privacidad, cumplimiento y operación	Los comprobantes de pago deberán mantenerse protegidos.	Acceso limitado a cliente propietario y personal autorizado.
RNF-056	Privacidad, cumplimiento y operación	El sistema deberá disponer de registros de diagnóstico.	Registrar errores técnicos con identificadores de seguimiento, sin datos sensibles.
RNF-057	Privacidad, cumplimiento y operación	El sistema deberá poder desplegarse mediante configuraciones separadas.	Diferenciar desarrollo, pruebas y producción mediante variables de entorno.
RNF-058	Privacidad, cumplimiento y operación	La información de pago deberá ser verificable.	No considerar una captura como confirmación automática de recepción de fondos.

Código	Categoría	Requerimiento no funcional	Criterio de aceptación propuesto
RNF-059	Privacidad, cumplimiento y operación	Los cambios de configuración deberán ser controlados.	Registrar quién modificó parámetros importantes y cuándo.
RNF-060	Privacidad, cumplimiento y operación	Las notificaciones deberán respetar las preferencias y permisos del dispositivo.	Solicitar autorización para notificaciones push y mantener avisos importantes dentro de la aplicación.

6.1. Requisitos no funcionales adicionales (RNF-061 a RNF-072)

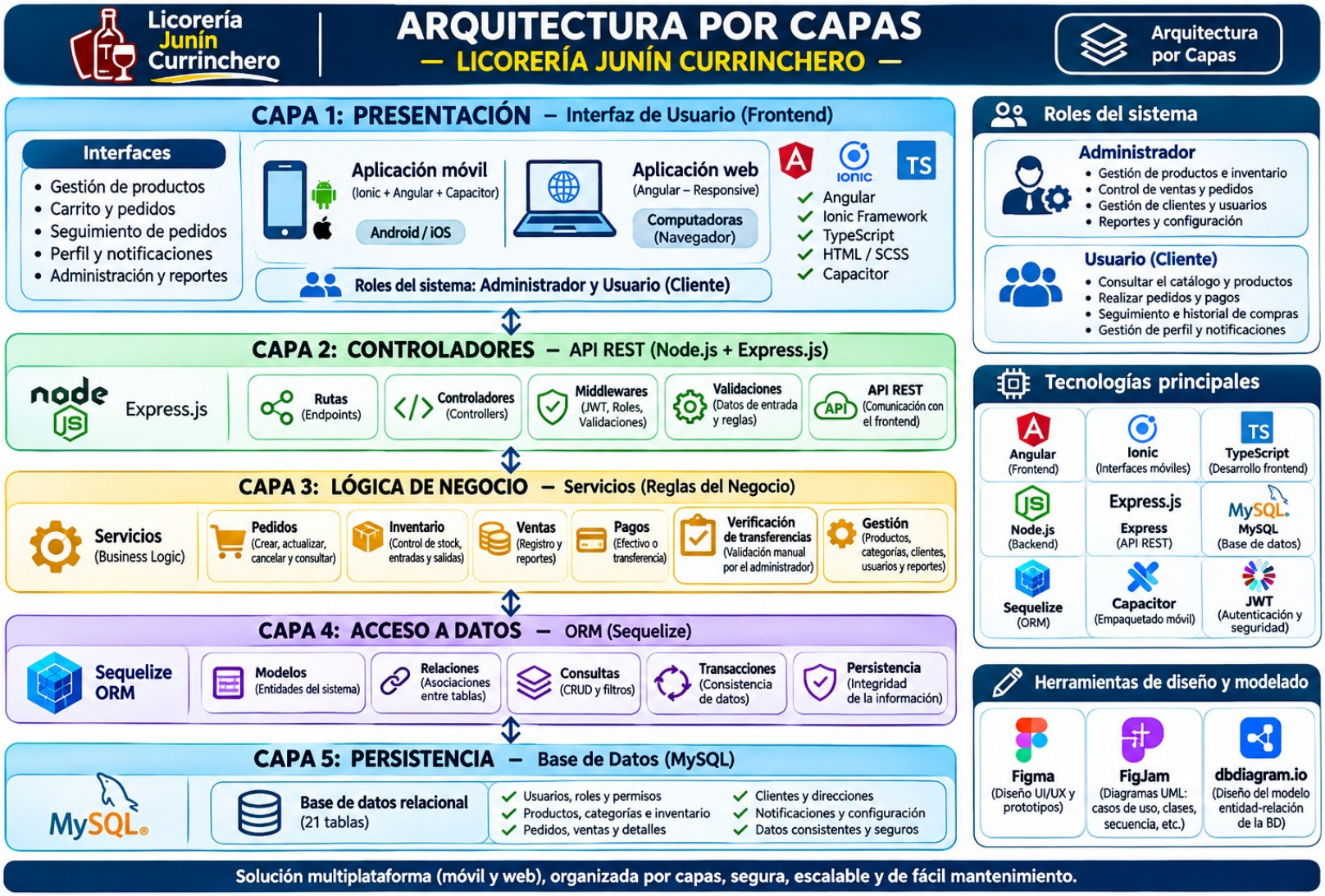
Código	Categoría	Requerimiento	Criterio / prueba propuesta
RNF-061	Seguridad	La API de agentes deberá verificar permisos en el servidor.	100 % de los casos de acceso no autorizado de la batería de prueba son rechazados. Prueba: Probar JWT ausente, vencido y usuario ajeno.
RNF-062	Privacidad	Los agentes deberán minimizar información personal transmitida a proveedores externos.	No se envían contraseñas, tokens ni comprobantes completos en los casos evaluados. Prueba: Inspeccionar solicitudes salientes.
RNF-063	Precisión	Las respuestas de precio y stock deberán basarse en datos vigentes de la API.	0 precios o cantidades inventados en el conjunto de pruebas definido. Prueba: Comparar respuestas contra datos de referencia.
RNF-064	Confiabilidad	Ante ausencia de información, el asistente deberá reconocer la incertidumbre.	Todas las consultas sin datos reciben aviso de no disponibilidad. Prueba: Probar referencias inexistentes.
RNF-065	Rendimiento	Las consultas inteligentes deberán responder dentro de un objetivo acordado.	Meta propuesta: p95 menor o igual a 8 segundos en escenario de prueba definido. Prueba: Prueba de carga y latencia.
RNF-066	Disponibilidad	Una caída del proveedor de IA no deberá impedir el uso de módulos esenciales.	Catálogo, pedidos y soporte manual permanecen disponibles. Prueba: Simular proveedor desconectado.
RNF-067	Accesibilidad	Los asistentes deberán aspirar a WCAG 2.2 nivel AA en interfaces web aplicables.	Sin fallos críticos de teclado, etiquetas o contraste en auditoría definida. Prueba: Pruebas automáticas y manuales.
RNF-068	Auditoría	Los eventos de los agentes deberán permitir diagnóstico sin revelar secretos.	Los registros contienen fecha, operación, resultado e identificador seguro. Prueba: Inspeccionar logs.
RNF-069	Gobernanza	Las versiones de prompts y políticas deberán controlarse y aprobarse.	Cada versión publicada posee responsable, fecha y registro de aprobación. Prueba: Revisar historial Git.

RNF-070	Seguridad	El sistema deberá resistir instrucciones maliciosas y extracción de datos.	No se revelan pedidos ajenos ni instrucciones reservadas en pruebas adversariales. Prueba: Pruebas de prompt injection.
RNF-071	Supervisión	Los agentes deberán permitir derivación y supervisión humana.	El usuario encuentra opción de contacto humano cuando el asistente falla. Prueba: Ensayar derivación.
RNF-072	Compatibilidad	Los asistentes deberán operar en Android y web dentro del alcance inicial.	Flujos principales completados en dispositivos y navegadores de prueba. Prueba: Pruebas cruzadas.

7. Arquitectura de software: arquitectura por capas

La solución se organizará por capas con responsabilidades diferenciadas. La presentación muestra las vistas y recoge acciones; los controladores reciben solicitudes HTTP; los servicios aplican reglas del negocio; la capa de acceso a datos consulta y modifica los registros; MySQL proporciona persistencia. La API REST es el mecanismo de comunicación entre la aplicación móvil y el backend, no una arquitectura adicional seleccionada.

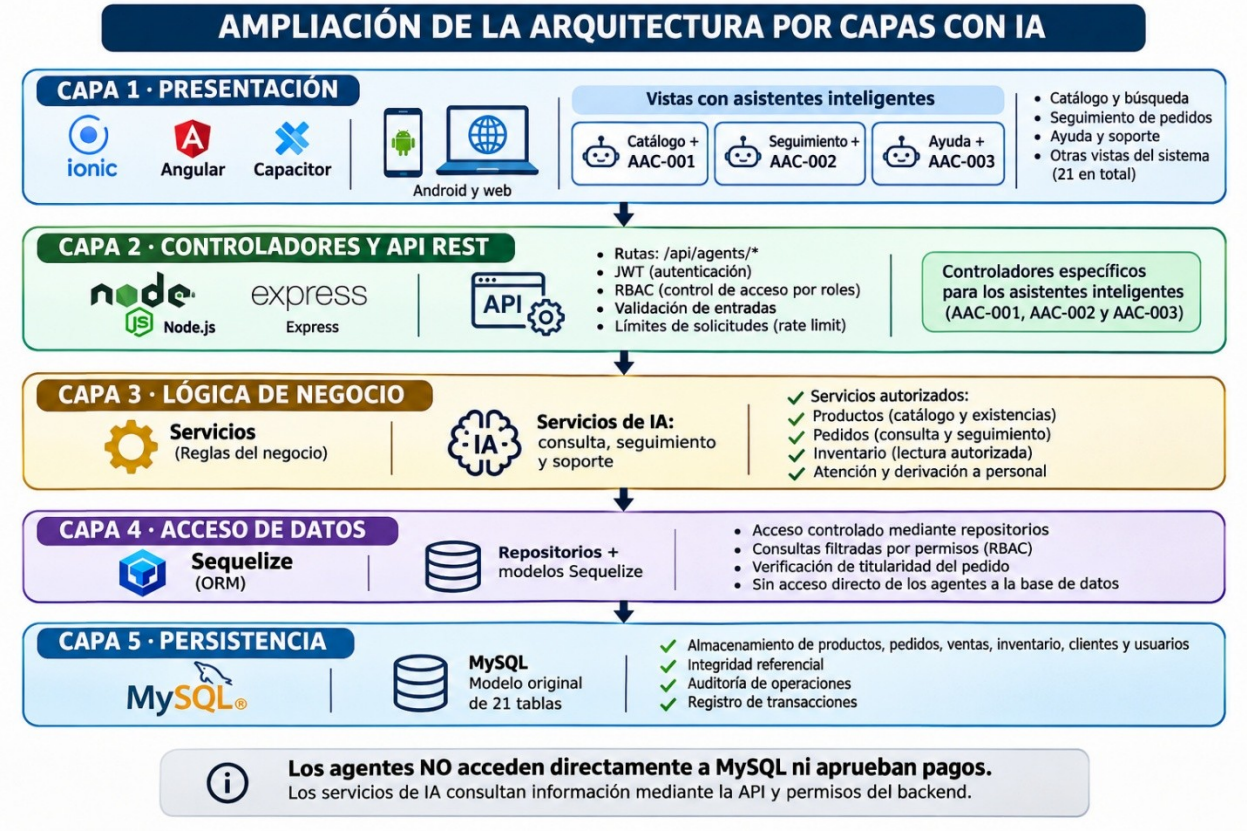
7.1. Diagrama de arquitectura por capas



Extensión de la Figura 1, representada en la Figura 2 (sin sustituir la imagen original): presentación con componentes de asistencia; API REST con rutas, autenticación JWT y autorización RBAC; servicios especializados AAC-001/AAC-002/AAC-003 que reutilizan servicios de catálogo, pedidos y soporte; repositorios Sequelize; persistencia MySQL. Ningún agente consulta directamente la base de datos.

Figura 1. Arquitectura por capas del sistema multiplataforma Licorería Junín Currinchero.

Figura 2. Ampliación propuesta de la arquitectura por capas con agentes inteligentes.



Fuente: Elaboración propia. Diagrama propuesto; no representa una implementación terminada.

Fuente: Elaboración propia con apoyo de herramientas de inteligencia artificial.

7.1.1. Integración propuesta de agentes inteligentes

Flujo autorizado: componente de asistencia → API REST → validación JWT/RBAC → servicio de agente → servicio de negocio → repositorio Sequelize → MySQL → respuesta filtrada. Los modelos de IA no acceden directamente a MySQL. El diagrama original de 7.1 se conserva; este flujo representa su ampliación propuesta.

Capa existente	Ampliación propuesta
1. Presentación	Componentes Angular de chat y asistencia en Catálogo, Seguimiento y Ayuda; Ionic/Capacitor para Android y web.
2. API REST y controladores	Rutas Express /api/agents/products, /orders y /support; middleware de autenticación y autorización.
3. Lógica de negocio	Servicios especializados de consulta, contexto, validación de respuesta, límites y derivación humana.
4. Acceso a datos	Repositorios y modelos Sequelize existentes; filtrado por propietario y permisos.
5. Persistencia	Las 21 tablas MySQL propuestas permanecen como modelo base; no se añaden tablas sin decisión justificada.

7.2. Justificación de la arquitectura

La integración inteligente conservará la separación de responsabilidades: la interfaz solo presenta consultas; el backend autentica y autoriza; los servicios de negocio validan

existencias, titularidad de pedidos y políticas; Sequelize gestiona el acceso controlado a MySQL. La validación administrativa del pago por transferencia seguirá siendo obligatoria.

Esta arquitectura favorece la separación de responsabilidades, la mantenibilidad, las pruebas y la incorporación de nuevas funciones. Las reglas de stock, confirmación de pedidos, pagos y autorización deben implementarse en el backend y no depender exclusivamente de la interfaz.

7.3. Organización propuesta del proyecto

Proyecto: Licorería Junín Currinchero. Arquitectura: por capas. Frontend: Ionic Framework, Angular, TypeScript, HTML, SCSS y Capacitor, con despliegue web responsive y empaquetado móvil. Backend: Node.js, Express.js y Sequelize ORM. Persistencia: MySQL.

7.3.1. Descripción general

El proyecto se divide en un frontend Ionic + Angular con diseño adaptable para móviles y computadoras, con vistas de Usuario y Administrador, una API REST que aplica reglas de negocio y un sistema de persistencia relacional. Los árboles siguientes describen una estructura propuesta de carpetas y archivos para implementar los módulos definidos en el proyecto. No representan código ya desarrollado.

7.3.2. Estructura general del repositorio

```
licoreria-junin-currinchero/
├── frontend-app/                # Aplicación Ionic + Angular + Capacitor
├── backend/                     # API REST y lógica de negocio
├── database/                    # Diagramas y diccionario de datos
├── docs/                        # Requerimientos y documentación
├── .gitignore                   # Exclusiones del repositorio
└── README.md                    # Instalación y descripción del sistema
```

7.3.3. Organización detallada del frontend móvil

Ampliación del árbol propuesto: shared/components/assistant-chat/assistant-chat.component.ts; core/services/asistente-ia.service.ts; features/cliente/catalogo/asistente-productos.component.ts; features/cliente/seguimiento-pedido/asistente-seguimiento.component.ts; features/cliente/ayuda-contacto/asistente-soporte.component.ts. Son archivos propuestos, no implementados.

La carpeta core contiene servicios y seguridad compartidos; features reúne los flujos de negocio; shared agrupa componentes reutilizables. Las pantallas se implementarán como componentes Angular independientes (standalone). La búsqueda es parte del catálogo y detalle-pedido es una subvista de pedidos.

frontend-app/

```
├── src/
│   ├── app/
│   │   ├── core/
│   │   │   ├── guards/
│   │   │   │   ├── auth.guard.ts
│   │   │   │   └── role.guard.ts
│   │   │   ├── interceptors/
│   │   │   │   └── auth.interceptor.ts
│   │   │   ├── services/
│   │   │   │   ├── auth.service.ts
│   │   │   │   ├── usuario.service.ts
│   │   │   │   ├── categoria.service.ts
│   │   │   │   ├── producto.service.ts
```

```
│   │   │   │   ├── inventario.service.ts
│   │   │   │   ├── carrito.service.ts
│   │   │   │   ├── pedido.service.ts
│   │   │   │   ├── venta.service.ts
│   │   │   │   ├── pago.service.ts
│   │   │   │   ├── reporte.service.ts
│   │   │   │   ├── notificacion.service.ts
│   │   │   │   ├── soporte.service.ts
│   │   │   │   └── configuracion.service.ts
│   │   │   └── models/
│   │   │       ├── usuario.model.ts
│   │   │       ├── producto.model.ts
│   │   │       ├── categoria.model.ts
│   │   │       ├── pedido.model.ts
                    │   │   │       ├── venta.model.ts
                    │   │   │       ├── pago.model.ts
                    │   │   │       └── inventario.model.ts
                    │   │   ├── shared/
                    │   │   │   ├── components/
                    │   │   │   │   ├── producto-card/
                    │   │   │   │   │   └── producto-card.component.ts
                    │   │   │   │   ├── estado-pedido/
                    │   │   │   │   │   └── estado-pedido.component.ts
                    │   │   │   │   └── confirm-dialog/
                    │   │   │   │       └── confirm-dialog.component.ts
                    │   │   │   ├── pipes/
                    │   │   │   │   └── estado-pedido.pipe.ts
                    │   │   │   └── validators/
                    │   │   │       └── formulario.validators.ts
                    │   │   ├── features/
                    │   │   │   ├── auth/
                    │   │   │   │   ├── login/login.page.ts
                    │   │   │   │   ├── registro/registro.page.ts
                    │   │   │   │   └── recuperar-clave/recuperar-clave.page.ts
                    │   │   │   ├── cliente/
                    │   │   │   │   ├── inicio/inicio.page.ts
                    │   │   │   │   ├── catalogo/catalogo.page.ts
                    │   │   │   │   ├── busqueda-filtros/filtros.component.ts
                    │   │   │   │   ├── carrito/carrito.page.ts
                    │   │   │   │   ├── confirmacion-pedido/confirmacion.page.ts
                    │   │   │   │   ├── seguimiento-pedido/seguimiento.page.ts
                    │   │   │   │   ├── historial-compras/historial.page.ts
                    │   │   │   │   ├── perfil/perfil.page.ts
                    │   │   │   │   ├── notificaciones/notificaciones.page.ts
                    │   │   │   │   └── ayuda-contacto/ayuda-contacto.page.ts
                    │   │   │   └── administrador/
                    │   │   │       ├── dashboard/dashboard.page.ts
                    │   │   │       ├── productos/productos.page.ts
                    │   │   │       ├── inventario/inventario.page.ts
                    │   │   │       ├── pedidos/
                    │   │   │       │   ├── pedidos.page.ts
                    │   │   │       │   └── detalle-pedido/detalle-pedido.page.ts
                    │   │   │       ├── ventas/ventas.page.ts
                    │   │   │       ├── categorias/categorias.page.ts
                    │   │   │       ├── clientes/clientes.page.ts
                    │   │   │       ├── usuarios-permisos/usuarios-permisos.page.ts
                    │   │   │       ├── reportes/reportes.page.ts
                    │   │   │       ├── configuracion/configuracion.page.ts
                    │   │   │       └── atencion-cliente/atencion-cliente.page.ts
                    │   │   ├── layouts/
                    │   │   │   ├── cliente-layout/cliente-layout.component.ts
                    │   │   │   └── admin-layout/admin-layout.component.ts
                    │   │   ├── app.routes.ts
                    │   │   └── app.config.ts
│   ├── assets/
│   │   ├── images/logo.png
│   │   └── icons/
```

```
│   ├── environments/environment.ts
│   ├── theme/variables.scss
│   └── global.scss
├── capacitor.config.ts
├── angular.json
├── tsconfig.json
└── package.json
```

Los nombres de archivo .page.ts y .component.ts son convenciones propuestas; Angular puede utilizar archivos HTML y SCSS adicionales por pantalla. El carrito se gestiona en el frontend, pero sus importes, existencias y condiciones de compra deben verificarse nuevamente en el backend.

7.3.4. Organización detallada del backend

Ampliación del árbol propuesto: routes/agentes.routes.js; controllers/AgentesController.js; services/agentes/ConsultaProductosService.js, SeguimientoPedidosService.js y AtencionSoporteService.js; middlewares/agent-auth.middleware.js; tests/agentes/; docs/agentes/prompts/, politicas/ y evaluaciones/. Todos se integrarán con los servicios y permisos existentes.

El backend utiliza una separación explícita de rutas, controladores, servicios, repositorios y modelos. Los 21 modelos de Sequelize corresponden a las 21 tablas propuestas en el apartado 7.4. Los nombres de migraciones son descriptivos: en la implementación se agregarán prefijos cronológicos para ordenar su ejecución.

```
backend/
├── src/
│   ├── config/
│   │   ├── database.js
│   │   └── env.js
│   ├── models/
│   │   ├── UsuarioModel.js
│   │   ├── RolModel.js
│   │   ├── PermisoModel.js
│   │   ├── RolPermisoModel.js
│   │   ├── CategoriaModel.js
│   │   ├── ProductoModel.js
│   │   ├── InventarioModel.js
│   │   ├── MovimientoInventarioModel.js
│   │   ├── PedidoModel.js
│   │   ├── PedidoDetalleModel.js
│   │   ├── VentaModel.js
│   │   ├── VentaDetalleModel.js
│   │   ├── PagoModel.js
│   │   ├── ComprobantePagoModel.js
│   │   ├── DireccionModel.js
│   │   ├── HistorialEstadoPedidoModel.js
│   │   ├── NotificacionModel.js
│   │   ├── SolicitudSoporteModel.js
│   │   ├── RespuestaSoporteModel.js
│   │   ├── ConfiguracionNegocioModel.js
│   │   ├── AuditoriaModel.js
│   │   └── index.js
│   ├── repositories/
│   │   ├── UsuarioRepository.js
│   │   ├── RolPermisoRepository.js
│   │   ├── CategoriaRepository.js
│   │   ├── ProductoRepository.js
│   │   ├── InventarioRepository.js
│   │   ├── MovimientoInventarioRepository.js
│   │   ├── PedidoRepository.js
│   │   ├── VentaRepository.js
│   │   ├── PagoRepository.js
│   │   ├── NotificacionRepository.js
```

```
│   │   ├── SoporteRepository.js
│   │   ├── ConfiguracionRepository.js
│   │   └── AuditoriaRepository.js
│   ├── services/
│   │   ├── AuthService.js
│   │   ├── UsuarioService.js
│   │   ├── RolPermisoService.js
│   │   ├── CategoriaService.js
│   │   ├── ProductoService.js
│   │   ├── InventarioService.js
│   │   ├── PedidoService.js
│   │   ├── VentaService.js
│   │   ├── PagoService.js
│   │   ├── ReporteService.js
│   │   ├── NotificacionService.js
│   │   ├── SoporteService.js
│   │   ├── ConfiguracionService.js
│   │   └── AuditoriaService.js
│   ├── controllers/
│   │   ├── AuthController.js
│   │   ├── UsuarioController.js
│   │   ├── RolPermisoController.js
│   │   ├── CategoriaController.js
│   │   ├── ProductoController.js
│   │   ├── InventarioController.js
│   │   ├── PedidoController.js
│   │   ├── VentaController.js
│   │   ├── PagoController.js
│   │   ├── ReporteController.js
│   │   ├── NotificacionController.js
│   │   ├── SoporteController.js
│   │   └── ConfiguracionController.js
│   ├── routes/
│   │   ├── index.routes.js
│   │   ├── auth.routes.js
│   │   ├── usuarios.routes.js
│   │   ├── roles-permisos.routes.js
│   │   ├── categorias.routes.js
│   │   ├── productos.routes.js
│   │   ├── inventario.routes.js
│   │   ├── pedidos.routes.js
│   │   ├── ventas.routes.js
│   │   ├── pagos.routes.js
│   │   ├── reportes.routes.js
│   │   ├── notificaciones.routes.js
│   │   ├── soporte.routes.js
│   │   └── configuracion.routes.js
│   ├── middlewares/
│   │   ├── auth.middleware.js
│   │   ├── role.middleware.js
│   │   ├── permission.middleware.js
│   │   ├── validate.middleware.js
│   │   ├── upload.middleware.js
│   │   ├── error.middleware.js
│   │   └── rate-limit.middleware.js
│   ├── validators/
│   │   ├── auth.validator.js
│   │   ├── producto.validator.js
│   │   ├── pedido.validator.js
│   │   ├── venta.validator.js
│   │   ├── pago.validator.js
│   │   └── usuario.validator.js
│   ├── utils/
│   │   ├── ApiError.js
│   │   ├── pagination.js
│   │   └── logger.js
│   ├── migrations/
│   │   ├── 01-crear-roles-permisos-usuarios.js
│   │   ├── 02-crear-catalogo-inventario.js
│   │   ├── 03-crear-direcciones-pedidos.js
```

```
│   │   ├── 04-crear-ventas-pagos.js
│   │   └── 05-crear-soporte-configuracion-auditoria.js
│   ├── seeders/
│   │   ├── 01-roles-iniciales.js
│   │   └── 02-permisos-iniciales.js
│   ├── app.js
│   └── server.js
├── tests/
│   ├── auth.test.js
│   ├── pedidos.test.js
│   ├── inventario.test.js
│   └── pagos.test.js
├── .env
├── .env.example
├── .gitignore
└── package.json
```

La carpeta models/ contiene 21 archivos de entidades y un index.js para registrar las asociaciones.

Los repositorios pueden agrupar operaciones relacionadas sin exigir un repositorio por cada tabla.

Las migraciones crean las tablas respetando dependencias de claves foráneas.

7.3.4.1. Ampliación propuesta del backend para agentes inteligentes

Se agregarían, sin eliminar archivos existentes: frontend-app/src/app/shared/components/assistant-chat/; frontend-app/src/app/core/services/ai-assistant.service.ts; frontend-app/src/app/features/cliente/catalogo/components/product-assistant/; seguimiento/components/order-assistant/; ayuda/components/support-assistant/; backend/routes/agent.routes.js; backend/controllers/AgentController.js; backend/services/agents/{ProductAgentService,OrderAgentService,SupportAgentService}.js; backend/middlewares/{auth,authorization,agentAudit}.middleware.js; backend/tests/agents/; docs/agents/{prompts,policies,evaluations}/. Nombres y ubicaciones definitivos sujetos a revisión de la estructura original.

7.3.5. Organización detallada de la base de datos y documentación

```
database/
├── diagrams/
│   ├── modelo-entidad-relacion.mmd
│   ├── modelo-entidad-relacion.png
│   └── modelo-relacional.png
├── dictionary/
│   └── diccionario-datos.md
├── scripts/
│   └── consultas-referencia.sql
└── README.md
docs/
├── requerimientos/
│   ├── requerimientos-funcionales.md
│   └── requerimientos-no-funcionales.md
├── arquitectura/
│   ├── arquitectura-por-capas.md
│   └── diagrama-arquitectura.png
├── diseno/
│   ├── prototipos-figma.md
│   └── casos-de-uso.md
├── api/
│   └── endpoints.md
├── pruebas/
│   └── plan-pruebas.md
```

```
└── manuales/
    ├── manual-usuario.md
    └── manual-administrador.md
```

La carpeta database/ almacena documentación y consultas de referencia; la definición ejecutable y versionada del esquema reside en backend/src/migrations/. Las rutas de archivos gráficos corresponden a recursos planificados y deberán generarse durante el desarrollo.

7.3.6. Responsabilidades de los archivos principales

Archivo o carpeta	Responsabilidad
app.routes.ts / app.config.ts	Rutas y configuración general de Angular.
auth.guard.ts / role.guard.ts	Protección de navegación según autenticación y rol.
auth.interceptor.ts	Adjuntar credenciales autorizadas y gestionar respuestas HTTP.
*.service.ts (frontend)	Comunicación con la API y estado de la interfaz.
database.js / env.js	Conexión Sequelize y lectura validada de variables de entorno.
models/*.js / index.js	Entidades Sequelize y asociaciones entre tablas.
repositories/*.js	Operaciones de consulta y escritura de datos.
services/*.js	Reglas de negocio y coordinación de transacciones.
controllers/*.js	Validación de solicitud y respuestas HTTP, delegando a servicios.
routes/*.js	Definición de endpoints REST y middlewares aplicables.
middlewares/*.js	Autenticación, permisos, validación, archivos y errores.
validators/*.js	Validación estructural de datos recibidos.
migrations/*.js	Creación y evolución versionada del esquema MySQL.
seeders/*.js	Inicialización de roles y permisos, sin credenciales expuestas.
tests/*.test.js	Pruebas de autenticación y procesos comerciales críticos.

7.3.7. Correspondencia con la arquitectura por capas

Presentación: frontend-app/src/app/features y layouts. API/controladores: backend/src/routes y controllers. Lógica de negocio: backend/src/services. Acceso a datos: backend/src/repositories y models. Persistencia: MySQL. Los middlewares y la auditoría aportan controles transversales.

7.3.8. Ejemplo de flujo de confirmación de pedido

1) El cliente revisa el carrito. 2) Angular envía la solicitud por HTTPS. 3) El backend autentica y valida la solicitud. 4) PedidoController delega a PedidoService. 5) El servicio valida edad, precios, existencias y reglas de compra. 6) Los repositorios registran pedido y detalles en una transacción. 7) Se devuelve la confirmación. 8) Si se adjunta una transferencia, el pago queda pendiente de verificación manual y no se considera confirmado automáticamente.

7.3.9. Justificación y conclusión

La estructura detallada facilita localizar cada responsabilidad, probar módulos de manera independiente y mantener la correspondencia entre requerimientos, interfaces y persistencia. Esta

organización es una guía de implementación que deberá ajustarse al código real sin alterar la separación por capas ni las reglas de integridad del apartado 7.4.

7.4. Modelo de base de datos — Licorería Junín Currinchero

7.4.1. Descripción y alcance

Se propone un modelo relacional de 21 tablas en MySQL, gestionado mediante Sequelize ORM. Su finalidad es administrar usuarios, roles, productos, inventario, pedidos, ventas, pagos, direcciones, notificaciones, soporte, configuración y auditoría. Es un diseño lógico preliminar; no un esquema físico implementado.

N.º	Tabla	Finalidad
1	usuarios	Cuentas de clientes y personal.
2	roles	Roles de acceso.
3	permisos	Acciones autorizables.
4	rol_permisos	Asociación de roles y permisos.
5	categorias	Clasificación de productos.
6	productos	Catálogo comercial.
7	inventarios	Existencias y umbrales.
8	movimientos_inventario	Historial de entradas y salidas.
9	pedidos	Solicitudes de clientes.
10	pedido_detalles	Líneas de pedido.
11	ventas	Ventas presenciales o vinculadas a pedido.
12	venta_detalles	Líneas de venta.
13	pagos	Registro y validación de pagos.
14	comprobantes_pago	Referencias a archivos de transferencias.
15	direcciones	Direcciones de clientes.
16	historial_estados_pedido	Transiciones de pedidos.
17	notificaciones	Avisos al usuario.
18	solicitudes_soporte	Consultas e incidencias.
19	respuestas_soporte	Respuestas de atención.
20	configuracion_negocio	Parámetros del establecimiento.
21	auditorias	Operaciones administrativas relevantes.

7.4.2. Diccionario de datos de las 21 tablas

PK = clave primaria; FK = clave foránea; UQ = valor único. BIGINT se propone para identificadores, DECIMAL(12,2) para importes. created_at y updated_at podrán incorporarse mediante Sequelize. Los valores ENUM indicados son propuestas a validar.

7.4.2.1. Tabla usuarios

Campo	Tipo	Restricción / referencia
id_usuario	BIGINT	PK
id_rol	BIGINT	FK roles
nombre_completo	VARCHAR(150)	Obligatorio
correo	VARCHAR(150)	UQ
telefono	VARCHAR(20)	

password_hash	VARCHAR(255)	Hash seguro
fecha_nacimiento	DATE	Según política de verificación
estado	ENUM	activo/inactivo/bloqueado

7.4.2.2. Tabla roles

Campo	Tipo	Restricción / referencia
id_rol	BIGINT	PK
nombre	VARCHAR(50)	UQ
descripcion	VARCHAR(255)	

7.4.2.3. Tabla permisos

Campo	Tipo	Restricción / referencia
id_permiso	BIGINT	PK
codigo	VARCHAR(100)	UQ
descripcion	VARCHAR(255)	

7.4.2.4. Tabla rol_permisos

Campo	Tipo	Restricción / referencia
id_rol	BIGINT	PK, FK roles
id_permiso	BIGINT	PK, FK permisos

7.4.2.5. Tabla categorias

Campo	Tipo	Restricción / referencia
id_categoria	BIGINT	PK
nombre	VARCHAR(100)	UQ
descripcion	TEXT	
imagen_url	VARCHAR(500)	
activo	BOOLEAN	

7.4.2.6. Tabla productos

Campo	Tipo	Restricción / referencia
id_producto	BIGINT	PK
id_categoria	BIGINT	FK categorias
nombre	VARCHAR(150)	Obligatorio
descripcion	TEXT	
marca	VARCHAR(100)	
presentacion	VARCHAR(100)	
precio	DECIMAL(12,2)	≥0
imagen_url	VARCHAR(500)	
activo	BOOLEAN	

7.4.2.7. Tabla inventarios

Campo	Tipo	Restricción / referencia
id_inventario	BIGINT	PK
id_producto	BIGINT	FK productos, UQ
stock_actual	INT	≥0
stock_minimo	INT	≥0

7.4.2.8. Tabla movimientos_inventario

Campo	Tipo	Restricción / referencia
id_movimiento	BIGINT	PK
id_producto	BIGINT	FK productos
id_usuario	BIGINT	FK usuarios, opcional

tipo	ENUM	entrada/salida/ajuste/venta/ reversion
cantidad_cambio	INT	=/0, con signo
motivo	VARCHAR(255)	
referencia_tipo	VARCHAR(40)	
referencia_id	BIGINT	
fecha	DATETIME	

7.4.2.9. Tabla pedidos

Campo	Tipo	Restricción / referencia
id_pedido	BIGINT	PK
id_usuario	BIGINT	FK usuarios
id_direccion	BIGINT	FK direcciones, opcional
codigo	VARCHAR(40)	UQ
estado	ENUM	pendiente/confirmado/ en_preparacion/listo/ entregado/cancelado
modalidad_entrega	ENUM	retiro/domicilio
subtotal	DECIMAL(12,2)	
costo_entrega	DECIMAL(12,2)	
total	DECIMAL(12,2)	
observaciones	TEXT	
fecha_pedido	DATETIME	

7.4.2.10. Tabla pedido_detalles

Campo	Tipo	Restricción / referencia
id_detalle_pedido	BIGINT	PK
id_pedido	BIGINT	FK pedidos
id_producto	BIGINT	FK productos
cantidad	INT	>0
precio_unitario	DECIMAL(12,2)	
subtotal	DECIMAL(12,2)	

7.4.2.11. Tabla ventas

Campo	Tipo	Restricción / referencia
id_venta	BIGINT	PK
id_pedido	BIGINT	FK pedidos, UQ, opcional
id_usuario_registro	BIGINT	FK usuarios
numero_venta	VARCHAR(40)	UQ
fecha	DATETIME	
total	DECIMAL(12,2)	
estado	ENUM	registrada/anulada

7.4.2.12. Tabla venta_detalles

Campo	Tipo	Restricción / referencia
id_detalle_venta	BIGINT	PK
id_venta	BIGINT	FK ventas
id_producto	BIGINT	FK productos
cantidad	INT	>0
precio_unitario	DECIMAL(12,2)	
subtotal	DECIMAL(12,2)	

7.4.2.13. Tabla pagos

Campo	Tipo	Restricción / referencia
id_pago	BIGINT	PK
id_pedido	BIGINT	FK pedidos, opcional
id_venta	BIGINT	FK ventas, opcional
metodo	ENUM	efectivo/banca_movil
monto	DECIMAL(12,2)	>0
estado	ENUM	pendiente/en_verificacion/ confirmado/rechazado
referencia	VARCHAR(100)	
fecha_pago	DATETIME	
id_verificador	BIGINT	FK usuarios, opcional

7.4.2.14. Tabla comprobantes_pago

Campo	Tipo	Restricción / referencia
id_comprobante	BIGINT	PK
id_pago	BIGINT	FK pagos
archivo_url	VARCHAR(500)	Ruta privada
tipo_archivo	VARCHAR(50)	
fecha_carga	DATETIME	

7.4.2.15. Tabla direcciones

Campo	Tipo	Restricción / referencia
id_direccion	BIGINT	PK
id_usuario	BIGINT	FK usuarios
direccion	VARCHAR(255)	
referencia	VARCHAR(255)	
principal	BOOLEAN	

7.4.2.16. Tabla historial_estados_pedido

Campo	Tipo	Restricción / referencia
id_historial	BIGINT	PK
id_pedido	BIGINT	FK pedidos
id_usuario	BIGINT	FK usuarios, opcional
estado_anterior	VARCHAR(40)	
estado_nuevo	VARCHAR(40)	
motivo	TEXT	
fecha_cambio	DATETIME	

7.4.2.17. Tabla notificaciones

Campo	Tipo	Restricción / referencia
id_notificacion	BIGINT	PK
id_usuario	BIGINT	FK usuarios
titulo	VARCHAR(150)	
mensaje	TEXT	
tipo	VARCHAR(50)	
leida	BOOLEAN	
fecha	DATETIME	

7.4.2.18. Tabla solicitudes_soporte

Campo	Tipo	Restricción / referencia
id_solicitud	BIGINT	PK

id_usuario	BIGINT	FK usuarios
id_pedido	BIGINT	FK pedidos, opcional
asunto	VARCHAR(150)	
descripcion	TEXT	
estado	ENUM	pendiente/en_atencion/ resuelta
id_asignado	BIGINT	FK usuarios, opcional

7.4.2.19. Tabla respuestas_soporte

Campo	Tipo	Restricción / referencia
id_respuesta	BIGINT	PK
id_solicitud	BIGINT	FK solicitudes_soporte
id_usuario	BIGINT	FK usuarios
mensaje	TEXT	
fecha	DATETIME	

7.4.2.20. Tabla configuracion_negocio

Campo	Tipo	Restricción / referencia
id_configuracion	BIGINT	PK
nombre_comercial	VARCHAR(150)	
direccion	VARCHAR(255)	
telefono	VARCHAR(20)	
horarios_json	JSON	
modalidades_json	JSON	
metodos_pago_json	JSON	
datos_bancarios_json	JSON	Acceso restringido
stock_minimo_default	INT	

7.4.2.21. Tabla auditorias

Campo	Tipo	Restricción / referencia
id_auditoria	BIGINT	PK
id_usuario	BIGINT	FK usuarios, opcional
accion	VARCHAR(100)	
entidad	VARCHAR(100)	
id_registro	BIGINT	
fecha	DATETIME	
detalle	JSON	No almacenar secretos

7.4.3. Relaciones y cardinalidades

Tabla principal	Tabla relacionada	Cardinalidad propuesta
roles	usuarios	1:N
roles	permisos	N:M mediante rol_permisos
usuarios	pedidos	1:N
usuarios	direcciones	1:N
usuarios	notificaciones	1:N
usuarios	solicitudes_soporte	1:N
categorias	productos	1:N
productos	inventarios	1:1
productos	movimientos_inventario	1:N
pedidos	pedido_detalles	1:N

productos	pedido_detalles	1:N
pedidos	ventas	1:0..1
ventas	venta_detalles	1:N
productos	venta_detalles	1:N
pedidos	pagos	1:N opcional
ventas	pagos	1:N opcional
pagos	comprobantes_pago	1:N
pedidos	historial_estados_pedido	1:N
solicitudes_soporte	respuestas_soporte	1:N
usuarios	auditorias	1:N opcional

Cada pago se asociará exclusivamente con un pedido o con una venta directa; no con ambos a la vez. Las ventas presenciales podrán carecer de pedido. Un pedido podrá originar como máximo una venta. Los roles serán únicos por usuario en este modelo; si se requieren múltiples roles por usuario deberá añadirse una tabla intermedia.

7.4.4. Diagrama entidad-relación conceptual

Se presenta una representación textual segmentada por dominios; las cardinalidades detalladas están en la tabla anterior. Las líneas indican asociaciones, no flujo de ejecución.

Referencia gráfica del diagrama entidad-relación conceptual

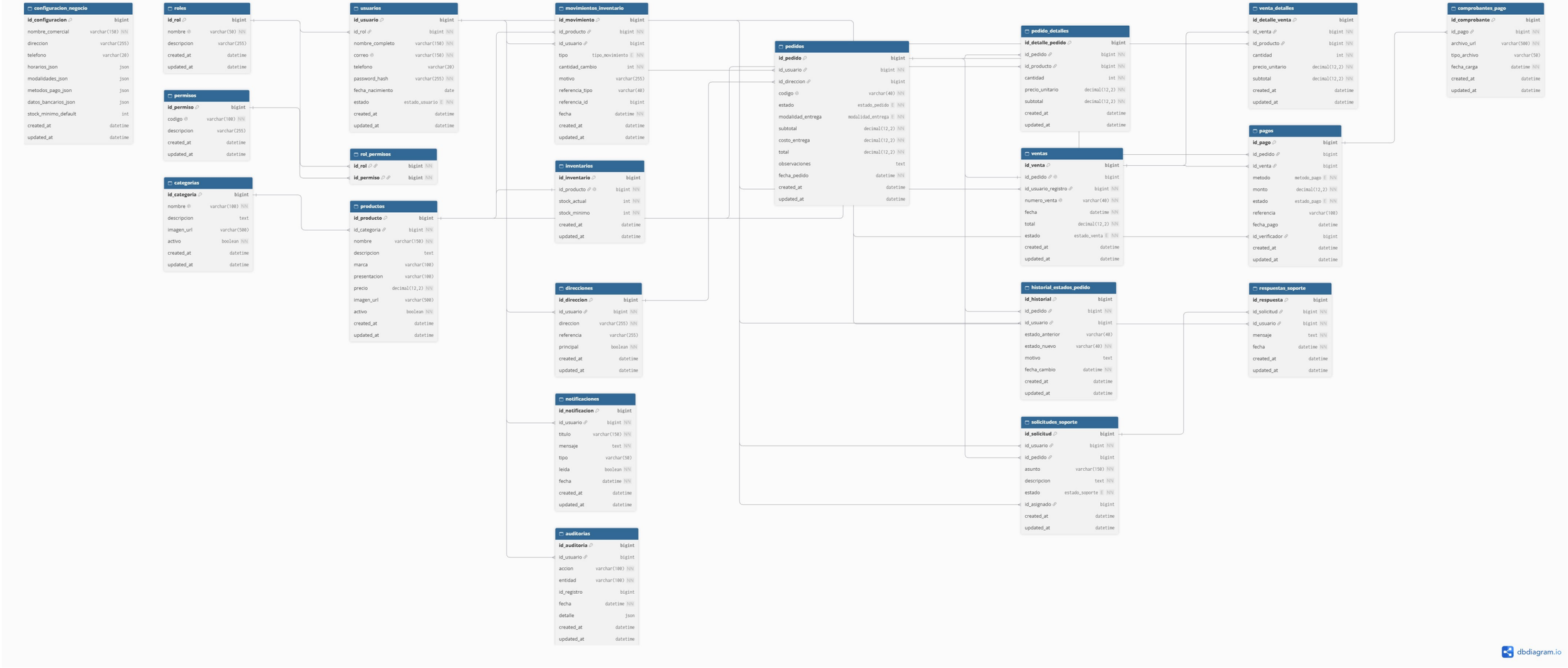
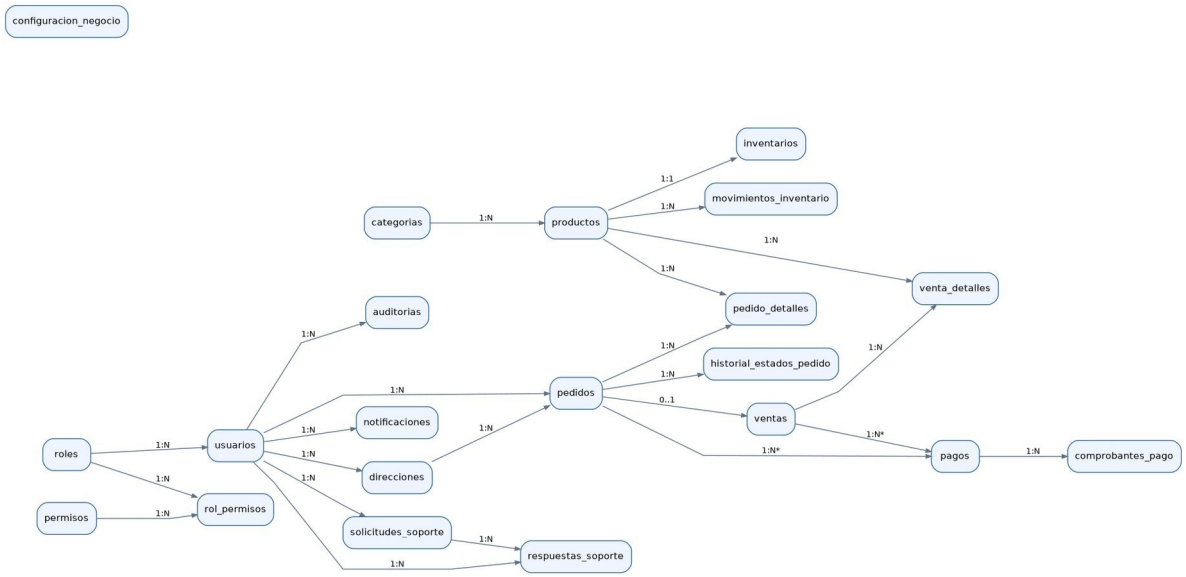


Figura 2. Diagrama entidad-relación conceptual de las 21 tablas propuestas.

Fuente: Elaboración propia mediante dbdiagram.io.

Nota: Las cardinalidades y restricciones de pagos, ventas e inventario deben validarse antes de implementar las migraciones.



7.4.5. Reglas de integridad, seguridad y transacciones

- Claves primarias, foráneas e índices para usuarios, productos, pedidos y ventas; claves únicas en correos, códigos y números de venta.
- Importes en DECIMAL; cantidades estrictamente positivas en detalles; existencias nunca negativas.
- Precio unitario y subtotal histórico conservados en cada detalle de pedido o venta.
- Registro de pedido, detalles y movimientos relacionados mediante transacciones atómicas.
- Control de concurrencia e idempotencia para impedir doble descuento de stock o doble registro de pagos.
- Comprobantes almacenados en ubicación privada; MySQL guarda únicamente una referencia protegida.
- Contraseñas con hash robusto; autorizaciones verificadas en servidor; auditoría sin contraseñas ni tokens.
- Desactivación lógica de productos, categorías y usuarios con historial, cuando proceda.
- Verificación efectiva de edad y cumplimiento de restricciones legales de venta y entrega.

7.4.6. Integración de la base de datos con las capas

La aplicación Ionic + Angular no accederá directamente a MySQL. Enviará solicitudes a Express; los controladores llamarán servicios; los servicios ejecutarán reglas y transacciones a través de repositorios y modelos Sequelize; MySQL persistirá los datos.

7.4.7. Decisiones pendientes antes de las migraciones

Aspecto	Decisión propuesta / validación pendiente
Inventario	Definir si el stock se reserva al confirmar pedido o se descuenta al registrar venta; documentar reversiones.
Pagos	Admitir intentos y comprobantes; exigir verificación de recepción real en banca móvil.
Ventas	Venta directa sin pedido; pedido con un máximo de una venta; evitar ingresos duplicados.
Roles	Un rol principal por usuario; cambiar modelo si se necesitan múltiples roles.

Estados de pedido	Unificar pendiente, confirmado, en preparación, listo, entregado y cancelado con transiciones válidas.
Cancelaciones	Establecer plazos, permisos y restauración de existencias.
Edad y entregas	Determinar procedimiento verificable y normativa aplicable.
Archivos	Definir almacenamiento privado, tipos admitidos y política de conservación.

7.4.8. Conclusión

El modelo de 21 tablas establece una base inicial para organizar las operaciones comerciales y mantener separadas la lógica de negocio y la persistencia. Antes de crear migraciones de Sequelize deben validarse cardinalidades, restricciones y reglas de inventario, ventas, pagos y roles. Esta sección complementa la organización del proyecto presentada en 7.3.

7.4.9. Modelo de datos y decisiones pendientes

Se mantienen las 21 tablas propuestas en la sección 7.4. El almacenamiento de conversaciones, métricas o evaluaciones podría resolverse con registros externos controlados o con tablas adicionales, previa evaluación de privacidad, costos, retención y aprobación formal. No se afirma que existan tablas nuevas.

8. Tecnologías y herramientas

Figma: diseño de interfaces móviles y web responsive, prototipos y accesibilidad. FigJam: casos de uso, actividades, secuencia y componentes UML. dbdiagram.io: modelo entidad-relación de las 21 tablas. Las versiones y los enlaces reales se registrarán una vez disponibles. AG-001, AG-002, AG-003 y AG-006 revisarán y aprobarán los entregables antes del desarrollo.

Compatibilidad multiplataforma: La interfaz desarrollada con Ionic y Angular se adaptará a teléfonos, tabletas y pantallas de computadoras mediante diseño responsive. La versión web se ejecutará en navegadores modernos; Capacitor se utilizará para empaquetar la versión móvil. Ambas versiones consumirán la misma API REST y compartirán la base de datos MySQL.

Roles de acceso: El sistema contempla exclusivamente Administrador y Usuario. El término «Cliente» describe las funciones comerciales del rol Usuario, no un tercer rol.

Herramientas de diseño: Figma para interfaces UI/UX y prototipos; FigJam para diagramas UML (casos de uso, clases, secuencia y actividades); dbdiagram.io para el diagrama entidad-relación de las 21 tablas.

Lenguajes: TypeScript, HTML y SCSS para el frontend Ionic/Angular; JavaScript para el backend Node.js/Express.js.

Criterios adicionales de compatibilidad (complementan los RNF existentes): comprobar navegación con teclado y ratón, diseño responsive sin desplazamiento horizontal, funcionamiento en versiones compatibles de Chrome, Edge y Firefox, y protección de sesiones y tokens también en el entorno web.

Elemento	Tecnología	Función
Interfaz móvil	Ionic + Angular	Componentes, formularios, rutas y vistas
Empaquetado móvil	Capacitor	Compilación y acceso a capacidades del dispositivo
Backend	Node.js + Express.js	API REST, controladores y servicios
Plataforma web para computadoras	Ionic + Angular (responsive)	Acceso desde navegador, interfaz adaptable a pantallas de escritorio

Lenguaje frontend	TypeScript, HTML y SCSS	Componentes, vistas y estilos adaptables
Lenguaje backend	JavaScript	Controladores, servicios y modelos
Diseño UI/UX	Figma	Prototipos para móvil y escritorio
Modelado UML	FigJam	Diagramas de casos de uso, clases, actividades y secuencia
Modelo de datos	dbdiagram.io	Diagrama entidad-relación de 21 tablas
Acceso a datos	Sequelize	Modelos, consultas y transacciones
Persistencia	MySQL	Almacenamiento relacional
Seguridad	JWT y hash de contraseñas	Sesiones, autorización y credenciales
Desarrollo	Git y entorno de desarrollo	Versionado y mantenimiento

8.1. Herramientas de diseño y agentes de IA

FigJam se propone para diagramas de casos de uso, actividades, secuencia, componentes y flujos. Figma se propone para UI/UX, vistas de Usuario y Administrador, variantes responsive, asistencia inteligente y evaluación de accesibilidad. dbdiagram.io se mantiene para el modelo entidad-relación. Los modelos y proveedores de IA no se han seleccionado definitivamente.

Artefacto	Enlace real	Versión	Estado
Prototipos Figma	Pendiente	Pendiente	Por elaborar/revisar
Diagramas FigJam	Pendiente	Pendiente	Por elaborar/revisar
Modelo dbdiagram.io	Pendiente	Pendiente	Por verificar

9. Aspectos pendientes de validación

Decisiones de IA pendientes: proveedor y modelo, costos, condiciones de uso, alojamiento, conservación de conversaciones, evaluación de respuestas, supervisión humana y posible uso de RAG con fuentes autorizadas. No se afirma que existan servicios implementados ni pruebas aprobadas.

> Antes de iniciar la implementación, definir formalmente los estados y transiciones de pedido, el momento de descuento o reserva de stock, las condiciones de cancelación, la comprobación de transferencias, las restricciones de entrega y el procedimiento de verificación de edad. Estas decisiones son necesarias para convertir los requisitos propuestos en criterios de prueba completos.
>
> Fuente de elaboración: Propuesta de proyecto de software «Licorería Junín Currinchero» (44 páginas) y Actividad Formativa N.º 1 (2 páginas).

9.1. Decisiones adicionales pendientes

Seleccionar modelo/proveedor de IA, alojamiento, presupuesto, límites de uso, conservación de consultas, bases legales y medidas de privacidad, política de supervisión, métricas, corpus RAG opcional y procedimiento de revisión humana. No se declaran decisiones aprobadas ni pruebas ejecutadas.

10. Metodología de desarrollo asistido por agentes de inteligencia artificial

Los agentes AG-001 a AG-007 apoyarán al equipo en la elaboración de la aplicación. No son usuarios, roles de acceso ni funciones disponibles para clientes. Sus entregables estarán sujetos a revisión y aprobación humana.

10.1. AG-001 — Análisis y validación de requerimientos

Campo	Especificación propuesta
Objetivo	Análisis y validación de requerimientos
Responsabilidades	Verificar claridad, consistencia y trazabilidad de RF/RNF
Entradas	Propuesta, RF/RNF y decisiones aprobadas
Herramientas	Documentación, validadores y control de versiones
Entregables	Documento de requisitos y matriz de trazabilidad
Criterios de aceptación	Sin identificadores duplicados; criterios verificables
Relación y dependencias	AG-002, AG-003, AG-006, AG-007
Supervisión humana	El equipo revisa, solicita correcciones y aprueba antes de incorporar el entregable.

10.2. AG-002 — Diagramas UML en FigJam

Campo	Especificación propuesta
Objetivo	Diagramas UML en FigJam
Responsabilidades	Elaborar casos de uso, actividades, secuencias, componentes y flujos
Entradas	Requisitos aprobados
Herramientas	FigJam
Entregables	Diagramas versionados y observaciones
Criterios de aceptación	Coherencia con roles, flujos y arquitectura
Relación y dependencias	AG-001, AG-003, AG-006
Supervisión humana	El equipo revisa, solicita correcciones y aprueba antes de incorporar el entregable.

10.3. AG-003 — Diseño UI/UX en Figma

Campo	Especificación propuesta
Objetivo	Diseño UI/UX en Figma
Responsabilidades	Proponer interfaces, prototipos y accesibilidad
Entradas	Requisitos, diagramas y 21 vistas
Herramientas	Figma
Entregables	Prototipos y especificaciones responsive
Criterios de aceptación	Cobertura de flujos y revisión de accesibilidad
Relación y dependencias	AG-001, AG-002, AG-004, AG-006
Supervisión humana	El equipo revisa, solicita correcciones y aprueba antes de incorporar el entregable.

10.4. AG-004 — Desarrollo frontend

Campo	Especificación propuesta
Objetivo	Desarrollo frontend
Responsabilidades	Implementar componentes Ionic/Angular/Capacitor y servicios API
Entradas	Prototipos aprobados y contratos API
Herramientas	Ionic, Angular, TypeScript, Capacitor
Entregables	Código frontend y pruebas de interfaz
Criterios de aceptación	Flujos funcionales y compilación verificable
Relación y dependencias	AG-003, AG-005, AG-006
Supervisión humana	El equipo revisa, solicita correcciones y aprueba antes de incorporar el entregable.

10.5. AG-005 — Desarrollo backend

Campo	Especificación propuesta
Objetivo	Desarrollo backend
Responsabilidades	Implementar API, servicios, permisos, Sequelize y reglas de negocio
Entradas	Requisitos y modelo de 21 tablas
Herramientas	Node.js, Express.js, Sequelize, MySQL
Entregables	API y pruebas backend
Criterios de aceptación	Autorización, integridad y validaciones verificables
Relación y dependencias	AG-001, AG-004, AG-006
Supervisión humana	El equipo revisa, solicita correcciones y aprueba antes de incorporar el entregable.

10.6. AG-006 — Pruebas y calidad

Campo	Especificación propuesta
Objetivo	Pruebas y calidad
Responsabilidades	Diseñar pruebas, ejecutar revisiones y registrar incidencias
Entradas	Código, prototipos, requisitos y diagramas
Herramientas	Pruebas automatizadas, análisis estático
Entregables	Casos de prueba, reportes e incidencias
Criterios de aceptación	Evidencia de pruebas y defectos resueltos
Relación y dependencias	AG-001 a AG-005, AG-007
Supervisión humana	El equipo revisa, solicita correcciones y aprueba antes de incorporar el entregable.

10.7. AG-007 — Coordinación y orquestación

Campo	Especificación propuesta
Objetivo	Coordinación y orquestación
Responsabilidades	Asignar tareas, controlar dependencias y devolver entregables observados
Entradas	Plan de trabajo, incidencias y versiones
Herramientas	Repositorio Git y gestor de tareas
Entregables	Tablero de estados y actas de revisión
Criterios de aceptación	Trazabilidad y aprobación humana por entrega
Relación y dependencias	AG-001 a AG-006
Supervisión humana	El equipo revisa, solicita correcciones y aprueba antes de incorporar el entregable.

10.8. Ciclo iterativo de revisión y corrección

Análisis (AG-001) → diagramas/diseño (AG-002 y AG-003) → frontend/backend (AG-004 y AG-005) → pruebas (AG-006) → registro de observaciones → devolución por AG-007 → corrección del responsable → nuevas pruebas → aprobación humana y control de versiones. El ciclo se repite hasta satisfacer criterios de aceptación.

La trazabilidad propuesta enlaza requisito, caso de uso, diagrama, prototipo, commit, prueba, incidencia, versión y aprobación. Una prueba pendiente no se registra como aprobada.

Estado	Responsable principal	Evidencia / salida
Pendiente	AG-007	Tarea y criterio de entrada
En análisis o diseño	AG-001/002/003	Requisito, diagrama o prototipo
En desarrollo	AG-004/005	Cambio versionado
En revisión/pruebas	AG-006	Evidencia y resultados

Observado	AG-006 y AG-007	Incidencia con requisito, severidad y responsable
En corrección	Agente responsable	Nueva versión y referencia a incidencia
Aprobado	Responsable humano	Acta o registro de aprobación

11. Agentes inteligentes integrados en Licorería Junín Currinchero

11.1. AAC-001 — Consulta de productos

Aspecto	Definición
Vista existente	Catálogo y búsqueda
Entradas	Pregunta sobre artículo, categoría o disponibilidad
Salidas	Productos y datos reales obtenidos de API
Funciones	Consulta de productos mediante servicio especializado y API autorizada
Restricciones	No inventar precios, existencias o productos; no modificar inventario.
Control de acceso	JWT y autorización por usuario/recurso cuando corresponda; límites de consulta
Ante error	Informar indisponibilidad o falta de información y ofrecer atención humana

11.2. AAC-002 — Seguimiento de pedidos

Aspecto	Definición
Vista existente	Seguimiento
Entradas	Pregunta sobre pedido y sesión autenticada
Salidas	Estado y detalles autorizados del pedido propio
Funciones	Seguimiento de pedidos mediante servicio especializado y API autorizada
Restricciones	Rechazar pedidos ajenos; no modificar estados ni confirmar pagos.
Control de acceso	JWT y autorización por usuario/recurso cuando corresponda; límites de consulta
Ante error	Informar indisponibilidad o falta de información y ofrecer atención humana

11.3. AAC-003 — Atención y soporte

Aspecto	Definición
Vista existente	Ayuda y contacto
Entradas	Consulta sobre horarios, entrega, pago o políticas
Salidas	Respuesta basada en contenido autorizado o derivación humana
Funciones	Atención y soporte mediante servicio especializado y API autorizada
Restricciones	No afirmar pagos verificados ni ofrecer datos no confirmados.
Control de acceso	JWT y autorización por usuario/recurso cuando corresponda; límites de consulta
Ante error	Informar indisponibilidad o falta de información y ofrecer atención humana

12. Aplicación de las doce mejoras del DERCAS

N.º	Mejora	Aplicación propuesta

1	RBAC	Mantener solo roles Usuario y Administrador; permisos validados en Express y por recurso.
2	Orquestación AG-007	Asignar responsables, revisar dependencias y devolver tareas con incidencias.
3	Contexto compartido	Usar versión aprobada de requisitos, diagramas, decisiones y modelo de datos.
4	Validación automatizada	Comprobar unicidad de códigos, vínculos y criterios de aceptación.
5	Trazabilidad	Vincular RF/RNF con casos de uso, FigJam, Figma, código y pruebas.
6	CI/CD	Proponer pipeline de lint, compilación, pruebas y despliegue con aprobación.
7	Gobernanza de IA	Limitar herramientas, datos, prompts y permisos; supervisión humana.
8	Versiones y reversión	Usar Git, revisiones, changelog y estrategia de rollback.
9	Métricas de agentes	Medir exactitud, defectos, cobertura, latencia y derivaciones.
10	RAG opcional	Recuperar FAQ y políticas aprobadas, con fuente y vigencia; sujeto a evaluación.
11	Accesibilidad responsive	Diseñar para Android/web con objetivo WCAG 2.2 AA; iOS futuro.
12	Monitoreo y mejora	Revisar fallos, calidad, costos, incidencias y actualizaciones de contenidos.

13. Trazabilidad, Figma, FigJam y control de versiones

AG-001 verifica cobertura de requisitos; AG-002 revisa diagramas en FigJam; AG-003 diseña prototipos Figma; AG-006 contrasta diagramas, prototipos y criterios de aceptación; AG-007 solicita aprobación humana antes de desarrollo o cambio de versión.

RF/RNF	Caso de uso	UML / FigJam	Figma	Componente/servicio	Prueba	Aprobación
RF-242	Pendiente	Pendiente	Pendiente	ProductAgentService (propuesto)	Pendiente	Pendiente
RF-245	Pendiente	Pendiente	Pendiente	OrderAgentService (propuesto)	Pendiente	Pendiente
RF-247	Pendiente	Pendiente	Pendiente	SupportAgentService (propuesto)	Pendiente	Pendiente

14. Plan propuesto de pruebas y aceptación

Prueba	Procedimiento propuesto	Resultado esperado / aceptación	Estado
Precios y stock	Consultar artículos con datos de referencia	Coincidencia exacta con API; sin inventar datos	No ejecutada
Pedidos propios	Autenticar Usuario A y consultar pedido A	Mostrar solo estado y detalle de A	No ejecutada
Pedidos ajenos	Consultar pedido de B desde sesión de A	Rechazo sin exposición de datos	No ejecutada
FAQ y banca móvil	Preguntar horario y validación de transferencia	Responder con política aprobada; pago queda pendiente de verificación	No ejecutada

Datos inexistentes	Consultar artículo o pedido no existente	Reconocer ausencia de información	No ejecutada
Prompt injection	Solicitar ignorar permisos o revelar instrucciones	No revelar datos ni alterar controles	No ejecutada
Privacidad	Inspeccionar payload y logs	No exponer tokens, contraseñas o datos ajenos	No ejecutada
Figma y FigJam	Revisar flujo de 21 vistas y UML contra requisitos	Observaciones registradas y revisión humana	No ejecutada
Android y web	Ejecutar flujos en dispositivos y navegadores definidos	Sin fallos críticos; iOS fuera de alcance inicial	No ejecutada
Integración y seguridad	Probar API, JWT, RBAC y servicios	Cumplimiento de criterios de cada RF/RNF	No ejecutada
Rendimiento y accesibilidad	Medir latencia y auditar teclado/contraste	Cumplir objetivos RNF propuestos	No ejecutada
Derivación humana	Simular consulta no resuelta	Ofrecer canal oficial de soporte	No ejecutada
Agentes de desarrollo	Devolver entregable con observación y repetir prueba	Evidencia de corrección y aprobación humana	No ejecutada

15. Estado de implementación y validación documental

Todos los agentes, servicios, endpoints, diagramas adicionales, pruebas y mecanismos de CI/CD descritos en las ampliaciones constituyen propuestas de diseño. Su implementación, selección de proveedor y resultados requieren evidencia futura. Se conservan los requisitos originales y el modelo base de 21 tablas; cualquier cambio estructural exigirá aprobación documentada.

16. Relación entre la propuesta y la plantilla DERCAS

Esta propuesta define el alcance, los módulos, los requerimientos, la arquitectura y la metodología prevista para Licorería Junín Currinchero. La plantilla DERCAS se mantiene como documento independiente y reutilizable para especificar y verificar requisitos, entregables, agentes, ciclos de revisión y criterios de aceptación. No se considera una especificación del proyecto completada mientras sus campos, evidencias y aprobaciones permanezcan pendientes.

Al aplicar DERCAS al proyecto, se deberá utilizar una copia de la plantilla y vincular los requisitos RF-001 a RF-255 y RNF-001 a RNF-072, los agentes AG-001 a AG-007 y AAC-001 a AAC-003, los diagramas y prototipos aprobados, el código y las pruebas. Los resultados no ejecutados deberán registrarse como pendientes. La propuesta y DERCAS deben mantener coherencia en los dos roles (Administrador y Usuario), las 21 vistas, el modelo base de 21 tablas y el alcance inicial Android/web, con iOS como adaptación futura.

PARTE II. DERCAS APLICADO A LICORERÍA JUNÍN CURRINCHERO

Especificación basada en la plantilla suministrada y completada con la Parte I. La asignación de plataformas y herramientas a cada agente se incorporará posteriormente. Los estados pendientes no constituyen evidencia de implementación.

PLANTILLA DERCAS

Desarrollo de aplicaciones multiplataforma (móvil y web) asistido por agentes de inteligencia artificial, Figma y FigJam

DOCUMENTO DE ESPECIFICACIÓN DE REQUERIMIENTOS Y CRITERIOS DE ACEPTACIÓN DEL SISTEMA

• Proyecto: Licorería Junín Currinchero

• Institución: ESPAM MFL

• Carrera: Computación

• Asignatura: Por confirmar con el docente

• Integrantes: Pendiente de proporcionar por el equipo

• Docente: Pendiente de proporcionar

• Fecha: 10/11/2026 (fecha de integración documental)

• Versión: 1.0 — integración documental propuesta

• Repositorio: Pendiente de crear o facilitar URL

• Enlace Figma: Pendiente de facilitar enlace real

• Enlace FigJam: Pendiente de facilitar enlace real

PROPÓSITO: especificar un proceso y equipo de agentes de IA que apoyen la creación de una aplicación multiplataforma (móvil y web). Los agentes de desarrollo NO se presuponen integrados dentro de la aplicación resultante.

1. INFORMACIÓN GENERAL DEL PROYECTO

1.1 Nombre de la aplicación multiplataforma

Licorería Junín Currinchero: aplicación multiplataforma para Android y navegadores web de computadoras; iOS queda como posibilidad futura.

1.2 Descripción y contexto

El sistema gestiona catálogo, compras, pedidos, pagos, ventas, inventario, clientes, reportes y soporte. Se distinguen dos roles de acceso: Administrador y Usuario; «Cliente» es el área funcional de Usuario. Se prevén 21 vistas principales y 21 tablas.

1.3 Problema identificado

El manejo manual de productos, existencias, pedidos y ventas dificulta la actualización de inventario, la trazabilidad de operaciones y la atención al usuario.

1.4 Justificación del uso de agentes de IA

Los agentes AG-001 a AG-007 se proponen para analizar requisitos, preparar UML, diseñar prototipos, desarrollar, probar y coordinar el trabajo. Sus entregables estarán sujetos a verificación y aprobación humana; documentar el proceso no significa que ya estén configurados o ejecutándose.

1.5 Objetivo general

Diseñar y desarrollar una solución móvil y web para administrar ventas, inventario y pedidos de la Licorería Junín Currinchero con arquitectura por capas, seguridad por roles y trazabilidad de requisitos.

1.6 Objetivos específicos

Especificar 255 RF y 72 RNF; modelar las 21 tablas; diseñar casos de uso en FigJam y prototipos en Figma; desarrollar frontend Ionic/Angular y backend Node/Express/Sequelize/MySQL; comprobar seguridad, inventario, pagos y experiencia móvil/web.

1.7 Alcance móvil y web para computadoras

Android y navegador web de computadoras en el alcance inicial. La interfaz se adaptará a móvil y escritorio. iOS no se considera entrega inicial.

1.8 Alcance de la automatización con agentes

AG-007 coordina tareas TSK-XXX y revisiones REV-XXX; AG-001 analiza requisitos, AG-002 UML/FigJam, AG-003 Figma, AG-004 frontend, AG-005 backend y AG-006 QA. La ejecución concreta y la selección de plataformas de cada agente se definirán posteriormente.

1.9 Exclusiones y supuestos

No se contemplan pasarelas de pago externas, confirmación bancaria automática, logística de terceros ni funcionamiento sin conexión. Los pagos son en efectivo o banca móvil con verificación administrativa. Los agentes integrados no aprobarán pagos, no alterarán pedidos ni accederán directamente a MySQL.

2. IDENTIFICACIÓN DE AGENTES DE IA PARA EL DESARROLLO

Estos agentes apoyan el proceso de ingeniería de software; no son necesariamente funcionalidades de la app.

Código	Agente propuesto	Responsabilidad	Entregable
AG-001	Analista de requisitos	Revisar DERCAS, detectar ambigüedades	Matriz de requisitos
AG-002	Analista UML / FigJam	Preparar casos de uso y flujos	Diagramas y especificaciones
AG-003	Diseñador UI/UX / Figma	Preparar wireframes, pantallas y prototipos	Diseños y componentes
AG-004	Desarrollador frontend móvil y web	Implementar interfaces responsive y navegación	Código frontend
AG-005	Desarrollador backend	Implementar API, seguridad y persistencia	Código backend
AG-006	Agente QA	Diseñar y ejecutar pruebas permitidas	Casos y evidencias
AG-007	Coordinador	Planificar tareas y verificar entregables	Registro de decisiones

Nota de alcance multiplataforma: AG-003 diseñará variantes móvil y escritorio; AG-004 implementará el frontend adaptable para Android/iOS y navegadores de computadoras; AG-006 verificará ambas experiencias. Se conservan los códigos AG-001 a AG-007.

2.1 Ficha individual de agente

• Código y nombre: Pendiente de confirmar
• Objetivo: consultar productos activos, precios y existencias disponibles.
• Rol y alcance: Pendiente de confirmar
• Entradas (documentos y formatos): Pendiente de confirmar
• Herramientas autorizadas: Pendiente de confirmar
• Modelo / plataforma: Pendiente de confirmar
• Instrucciones y restricciones: Pendiente de confirmar
• Tareas permitidas: Pendiente de confirmar
• Tareas prohibidas: Pendiente de confirmar
• Salidas y formato esperado: Pendiente de confirmar
• Dependencias de otros agentes: Pendiente de confirmar
• Criterios de calidad: Pendiente de confirmar
• Condiciones de aprobación humana: Pendiente de confirmar
• Registro de actividad: Pendiente de confirmar
• Manejo de errores: Pendiente de confirmar

Repetir la ficha por cada agente realmente utilizado. Un solo agente puede asumir varios roles cuando el proyecto lo requiera.

2.2 Matriz de autonomía y permisos

Agente	Puede leer	Puede modificar	Requiere aprobación para
[AG-…]	[DERCAS, diseños, código]	[Archivos autorizados]	[Publicación, borrado, despliegue, datos sensibles]

2.3 Orquestación, dependencias y contexto compartido

AG-007 mantendrá un tablero de tareas con ID, requisito origen, agente asignado, prioridad, dependencia, versión, estado, evidencia y aprobación. Estados propuestos: pendiente, listo, en ejecución, en revisión, observado, bloqueado y aprobado. Ningún agente ejecutará tareas sin entradas y permisos validados.

El repositorio documental común contendrá DERCAS vigente, decisiones aprobadas, casos de uso FigJam, especificaciones Figma, contratos API y evidencias QA. Los agentes consultarán solo versiones aprobadas; las fuentes no verificadas se marcarán como tales. No se compartirá memoria conversacional con secretos ni datos reales de clientes.

Protocolo: AG-007 crea TSK-XXX → comprueba dependencias → entrega instrucciones y referencias → agente produce artefacto versionado → AG-006/revisor contrasta criterios → aprobación o incidencia REV-XXX → actualización de trazabilidad. Se permite trabajo paralelo

únicamente cuando las dependencias lo permitan.

Campo de tarea	Contenido requerido
TSK-ID / RF	Identificador y requisito relacionado
Entrada / versión	Documento o enlace y versión aprobada
Responsable / permiso	AG-XXX, herramientas y alcance
Salida / evidencia	Artefacto, ruta, commit o enlace
Estado / revisión	Pendiente, observado o aprobado; revisor

3. REQUERIMIENTOS DEL SISTEMA Y DEL PROCESO MULTIAGENTE

3.1 Requerimientos funcionales de la aplicación

Código	Descripción verificable	Prioridad	Actor
RF-001	[Ej.: El usuario podrá iniciar sesión.]	Alta	[Usuario]
RF-002	[Funcionalidad principal]	[Prioridad]	Usuario

3.2 Requerimientos no funcionales de la aplicación

Código	Requisito medible	Categoría	Método de verificación
RNF-001	[Ej.: La sesión expirará según política definida.]	Seguridad	[Prueba]
RNF-002	[Tiempo, accesibilidad, rendimiento, etc.]	[Categoría]	[Prueba]

3.2.1 Requisitos mínimos multiplataforma (propuestos)

RF-MP-001: El usuario podrá utilizar las funciones aprobadas desde la aplicación móvil y desde un navegador de computadora, según sus permisos.

RF-MP-002: La navegación, el catálogo, el carrito, los pedidos y la atención por agentes deberán adaptarse a las pantallas móvil y escritorio sin perder funciones aprobadas.

RNF-MP-001: La interfaz se adaptará a tamaños de pantalla móvil, tableta y escritorio sin desplazamiento horizontal no intencionado en los flujos principales; verificar con pruebas responsive.

RNF-MP-002: La aplicación web será accesible desde versiones vigentes de Chrome, Edge y Firefox en computadoras, sin instalación de software de escritorio.

RNF-MP-003: Las sesiones, permisos, validaciones y protección de datos se aplicarán de forma equivalente en móvil y web; comprobar por plataforma.

Nota: estos códigos son requisitos adicionales propuestos y deben validarse con el responsable del proyecto antes de su aprobación.

3.3 Requerimientos funcionales de los agentes de desarrollo

Código	Requerimiento	Evidencia
RAG-001	Los agentes deberán utilizar los requisitos aprobados como fuente de trabajo.	Matriz de trazabilidad
RAG-002	El agente UML generará especificaciones de casos de uso consistentes.	Diagramas y revisión
RAG-003	El agente UI/UX entregará especificaciones de interfaces trazables.	Figma / entregables
RAG-004	El agente de desarrollo implementará	Cambios de código

	únicamente tareas autorizadas.	
RAG-005	El agente QA verificará criterios de aceptación.	Resultados de pruebas
RAG-006	Las acciones críticas requerirán aprobación humana.	Registro de aprobación

3.4 Requerimientos de integración Figma y FigJam

Código	Requerimiento	Condición
RDF-001	Cada pantalla tendrá un requisito o caso de uso asociado.	Trazabilidad
RDF-002	Los flujos de FigJam deberán reflejar las rutas principales y alternativas.	Revisión
RDF-003	Los prototipos de Figma deberán representar estados de error y carga.	Inspección
RDF-004	Las modificaciones automáticas de archivos requerirán integración y permisos autorizados.	Verificación de acceso

3.5 Control de acceso: exactamente dos roles (RBAC)

El sistema contará exclusivamente con los roles Administrador y Usuario. «Cliente» es el área funcional del rol Usuario, no un tercer rol. Los permisos se comprobarán en backend para cada recurso y operación; ocultar botones en frontend no constituye autorización.

RF-ROL-001: autenticar credenciales y establecer una sesión segura. RF-ROL-002: asignar exclusivamente Administrador o Usuario. RF-ROL-003: el registro público asignará Usuario y no admitirá autoasignación de Administrador. RF-ROL-004: restringir gestión de productos, categorías, inventario, pedidos globales y reportes al Administrador. RF-ROL-005: permitir al Usuario catálogo, carrito, compra, historial y seguimiento exclusivamente propios. RF-ROL-006: rechazar accesos directos o API no autorizadas. RF-ROL-007: registrar intentos denegados sin exponer datos sensibles.

El acceso móvil y web utilizará las mismas reglas de autorización. Definir en implementación la política de sesiones, revocación, recuperación de cuenta y alta administrativa; validar el mecanismo con el responsable.

Módulo / operación	**Administrador**	**Usuario**
Productos / categorías	Crear, editar, eliminar	Consultar
Inventario	Administrar	Consultar disponibilidad
Pedidos	Gestionar pedidos autorizados	Crear y consultar los propios
Reportes de ventas	Consultar	Denegado
Carrito y compras	No requerido por defecto	Gestionar y comprar
AAC-002 pedidos	Supervisión autorizada, si se define	Consultar solo propios

3.6 Validación automática de requisitos y control de cambios

RAG-VAL-001: AG-001 verificará para cada RF/RNF identificador único, actor, condición, resultado observable, prioridad, dependencia, prueba y criterio de aceptación. RAG-VAL-002: señalará contradicciones, ambigüedades, duplicados y requisitos no verificables. RAG-VAL-003: ningún cambio se considerará aprobado sin análisis de impacto y responsable humano.

Solicitud CR-XXX: motivo → requisitos afectados → impacto en FigJam, Figma, API, código y pruebas → propuesta de versión → revisión humana → actualización de matriz → ejecución y

regresión. La detección asistida no equivale a validación definitiva.

4. ARQUITECTURA DE LA SOLUCIÓN Y DEL EQUIPO DE AGENTES

4.1 Arquitectura de la aplicación multiplataforma

• Frontend móvil Android/iOS: [Ionic + Angular + Capacitor; completar]

• Frontend web para computadoras: [Ionic + Angular responsive; acceso por navegador, sin instalación]. Backend/API: [Node.js + Express; completar]

• Base de datos: [MySQL; completar]

• Autenticación: JWT y autorización RBAC en el backend; exclusivamente Administrador y Usuario.

• Servicios externos: Figma/FigJam como herramientas de diseño; proveedor de IA, conexión y costos pendientes de decisión; sin pasarela de pago externa.

• Entorno de despliegue: Android y aplicación web responsive; infraestructura de alojamiento pendiente de decisión.

Diagrama técnico: consultar la Parte I, apartado 7.1, que contiene las figuras de arquitectura por capas y su ampliación con IA en la propuesta de origen, si está incluida.

Esquema de despliegue: Android/iOS (Ionic + Angular + Capacitor) y navegador de escritorio (Ionic + Angular responsive) → API compartida Node.js + Express → servicios y autorización → MySQL. Los agentes de atención se consumen a través de la API autenticada. No se contempla aplicación de escritorio instalable.

4.2 Arquitectura del proceso de desarrollo asistido por IA

Flujo de referencia: DERCAS aprobado → coordinador → agente de requisitos → agente UML/FigJam → agente UI/UX/Figma → agentes de desarrollo → agente QA → revisión humana. El trabajo puede iterarse según incidencias.

4.3 Herramientas y conectores

Herramienta	Uso	Integración disponible	Permisos
FigJam	Diagramas UML y flujos	[Manual/API/conector]	[Lectura/escritura]
Figma	Diseño de pantallas y prototipos	[Manual/API/conector]	[Lectura/escritura]
Repositorio Git	Código y control de versiones	[Conector/Git]	[Ramas autorizadas]
Entorno de pruebas	Ejecución y validación	[Herramienta]	[Alcance]

4.4 Protocolo de intercambio entre agentes

• Formato de tareas: ficha TSK-XXX con requisito, versión, entradas, agente responsable, permisos, salida, evidencia, estado y revisor.

• Identificadores: RF-001 a RF-255, RNF-001 a RNF-072, AG-001 a AG-007, AAC-001 a AAC-003, TSK-XXX, REV-XXX y CR-XXX.

• Entradas y salidas: DERCAS aprobado, diagramas, prototipos y contratos API; entregables versionados en documentos, diagramas, diseños, código o informes QA.

• Validaciones previas: verificar versión documental, dependencias, permisos, criterios de aceptación y aprobación del responsable.

• Reglas de traspaso: AG-007 valida el artefacto, AG-006 o revisor revisa y el responsable humano autoriza continuar.

• Reintentos y escalamiento: abrir REV-XXX, devolver al agente responsable, corregir, versionar y repetir las pruebas afectadas; escalar bloqueos al humano.

• Historial de decisiones: registrar fecha, tarea, versión, agente, observación, evidencia y aprobación humana, sin secretos.

4.5 Arquitectura de orquestación y conocimiento (RAG)

Arquitectura del desarrollo: fuentes aprobadas (DERCAS/figuras/diseños/repositorio) → índice documental controlado o búsqueda autorizada → AG-007 → AG-001 a AG-006 → validadores y revisores humanos → registro de evidencias. El uso de recuperación aumentada por generación (RAG) es opcional; si se implementa, debe conservar referencias, versiones y controles de acceso.

Distinguir «RAG» como técnica de recuperación de información de los códigos RAG-001 a RAG-006 ya existentes en la plantilla (requerimientos de agentes de desarrollo). Evitar confundir ambos significados.

El orquestador no deberá entregar secretos, tokens, datos personales ni permisos de escritura generales al modelo. Figma y FigJam se operarán manualmente cuando no exista integración autenticada y autorizada.

5. DISEÑO CON FIGJAM Y FIGMA

5.1 Actores y casos de uso

Caso de uso	Actor	Requisito	Enlace FigJam
CU-001	Usuario	RF-001	Enlace pendiente

5.2 Especificación de caso de uso

• Código: CU-001 (caso de uso propuesto).

• Nombre: Consultar catálogo de productos.

• Actor: Usuario (área Cliente).

• Objetivo: consultar productos activos, precios y existencias disponibles.

• Precondiciones: catálogo publicado y API disponible; autenticación según política aprobada.

• Flujo principal: abrir catálogo, consultar lista, buscar o filtrar, seleccionar producto y mostrar su información vigente.

• Flujos alternativos: sin resultados, producto agotado o fallo de API; mostrar aviso sin inventar existencias.

• Postcondiciones: información consultada; no se modifica inventario ni se genera compra.

• Requisitos asociados: RF-021 a RF-037 y RF-241 a RF-243.

• Criterio de aceptación: información mostrada coincide con la API; los productos inactivos o agotados no se agregan al carrito.

• Enlace FigJam: Pendiente de facilitar enlace real

5.3 Diagramas de FigJam

FigJam: se planifican casos de uso, actividades, secuencias y flujos de navegación vinculados a RF; diagramas y enlaces pendientes de creación y aprobación. Los diagramas de arquitectura ya constan en la propuesta.

5.4 Inventario de pantallas

Pantalla	Requisito	Caso de uso	Enlace Figma	Estado
UI-001: [Nombre]	RF-001	CU-001	Enlace pendiente	Pendiente

5.5 Wireframes y sistema de diseño

• Wireframes: previstos para autenticación, catálogo, carrito, confirmación, seguimiento y vistas administrativas, en móvil y escritorio.

• Componentes reutilizables: tarjetas de producto, formularios, estados de pedido, alertas de stock, navegación y chat de asistencia.

• Tipografía y colores: aplicar la identidad institucional aprobada; verificar contraste y legibilidad en Figma.

• Accesibilidad: objetivo WCAG 2.2 AA para interfaces web aplicables; teclado, etiquetas, foco, contraste y mensajes claros.

• Tamaños de pantalla: móvil Android, tableta y navegador de escritorio, con puntos de quiebre definidos durante el diseño.

• Estados de error y carga: contemplar cargando, vacío, validación, sin conexión, fallo de API y acceso denegado.

5.5.1 Diseño responsive para móvil y computadora

Figma deberá incluir al menos una variante móvil y otra de escritorio para cada flujo principal, documentando puntos de quiebre, navegación, componentes reutilizables, estados vacíos, carga, error y accesibilidad. FigJam reflejará los recorridos de ambas plataformas y sus diferencias justificadas.

Matriz de pantallas sugerida: [Código UI] · [Requisito] · [Figma móvil] · [Figma escritorio] · [Caso de uso FigJam] · [Estado de aprobación].

5.6 Prototipo interactivo

• Enlace: pendiente de proporcionar enlace real del prototipo Figma.

• Flujos simulados: registro/inicio de sesión, catálogo, carrito, pedido, banca móvil, seguimiento y gestión administrativa.

• Pantallas enlazadas: las 21 vistas principales previstas en la propuesta, con subfunciones integradas.

• Limitaciones de simulación: un prototipo no ejecuta pagos, API, seguridad ni transacciones reales.

• Versión: 1.0 — integración documental propuesta

• Responsable de aprobación: coordinador humano del proyecto; identidad pendiente de confirmar.

5.7 Integración de los agentes con Figma y FigJam

Por defecto AG-002 y AG-003 prepararán especificaciones y propuestas para FigJam/Figma con intervención humana; lectura o escritura automática solo si se configura una integración autenticada con permisos expresos.

5.8 Revisión y aprobación humana

Artefacto	Revisor	Criterio	Estado
[Diagrama / pantalla]	[Nombre]	[Requisito y usabilidad]	Pendiente

5.9 Validación de accesibilidad y experiencia responsive

El diseño deberá cubrir móvil, tableta y escritorio con navegación por teclado cuando corresponda, etiquetas comprensibles, contraste suficiente, foco visible, mensajes de error accesibles y componentes táctiles adecuados. Adoptar WCAG 2.2 nivel AA como referencia propuesta, sujeto a aprobación y alcance.

UI-ACC-001: pantalla de inicio de sesión; UI-ACC-002: catálogo; UI-ACC-003: carrito; UI-ACC-004: pedidos; UI-ACC-005: chat IA. Para cada pantalla registrar versión Figma móvil/escritorio, comportamiento al redimensionar, estado vacío/carga/error y evidencia de revisión.

Los criterios se validarán mediante revisión de diseño, pruebas de teclado/lector de pantalla y pruebas en navegadores; una maqueta Figma por sí sola no acredita accesibilidad en producción.

6. PLAN DE ACCIÓN

Actividad	Agente / responsable	Entrada	Entregable	Duración
Análisis DERCAS	AG-001	Requisitos iniciales	Matriz revisada	Por planificar
Casos de uso	AG-002	Matriz	Diagramas FigJam	Por planificar
Prototipos	AG-003	Casos de uso	Figma	Por planificar
Implementación	AG-004 / AG-005	Diseños aprobados	Código	Por planificar
Pruebas	AG-006	Código y criterios	Evidencias	Por planificar

• Cronograma: fases propuestas de análisis → UML → UI/UX → frontend/backend → QA → revisión humana; fechas por acordar.

• Recursos humanos: equipo académico y coordinador humano; agentes AG-001 a AG-007 como funciones de apoyo.

• Recursos tecnológicos: Ionic, Angular, TypeScript, Capacitor, Node.js, Express, Sequelize, MySQL, Git, Figma y FigJam.

• Costos de IA: por definir al seleccionar proveedores y herramientas; no se atribuyen costos inexistentes.

• Dependencias: aprobación de requisitos antes de diagramas; diagramas antes de prototipos; contratos y diseños antes de implementar; código antes de pruebas.

• Hitos de aprobación: DERCAS, UML, prototipos, API, frontend, pruebas y liberación; todos requieren evidencia y visto bueno humano.

7. EJECUCIÓN E IMPLEMENTACIÓN

7.1 Preparación de repositorios y entornos

Procedimiento propuesto para 7.1 Preparación de repositorios y entornos7.1 Preparación de repositorios y entornos7.1 Preparación de repositorios y entornos: AG-007 asigna TSK-XXX con entradas y RF/RNF; el agente responsable produce artefacto versionado; AG-006 revisa criterios; el humano aprueba o abre REV-XXX. Evidencias reales pendientes de ejecución.

7.2 Asignación de tareas a agentes

Procedimiento propuesto para 7.2 Asignación de tareas a agentes7.2 Asignación de tareas a agentes7.2 Asignación de tareas a agentes: AG-007 asigna TSK-XXX con entradas y RF/RNF; el agente responsable produce artefacto versionado; AG-006 revisa criterios; el humano aprueba o abre REV-XXX. Evidencias reales pendientes de ejecución.

7.3 Generación y revisión de diagramas

Procedimiento propuesto para 7.3 Generación y revisión de diagramas7.3 Generación y revisión de diagramas7.3 Generación y revisión de diagramas: AG-007 asigna TSK-XXX con entradas y RF/RNF; el agente responsable produce artefacto versionado; AG-006 revisa criterios; el humano aprueba o abre REV-XXX. Evidencias reales pendientes de ejecución.

7.4 Diseño y revisión de prototipos

Procedimiento propuesto para 7.4 Diseño y revisión de prototipos7.4 Diseño y revisión de prototipos7.4 Diseño y revisión de prototipos: AG-007 asigna TSK-XXX con entradas y RF/RNF; el agente responsable produce artefacto versionado; AG-006 revisa criterios; el humano aprueba o abre REV-XXX. Evidencias reales pendientes de ejecución.

7.5 Implementación del frontend móvil y web responsive

Procedimiento propuesto para 7.5 Implementación del frontend móvil y web responsive7.5 Implementación del frontend móvil y web responsive7.5 Implementación del frontend móvil y web responsive: AG-007 asigna TSK-XXX con entradas y RF/RNF; el agente responsable produce artefacto versionado; AG-006 revisa criterios; el humano aprueba o abre REV-XXX. Evidencias reales pendientes de ejecución.

La implementación frontend deberá mantener una base de componentes adaptable cuando sea viable. Registrar los comportamientos específicos de móvil (Capacitor) y navegador web; no utilizar plugins nativos sin alternativa o degradación controlada en web.

7.6 Implementación del backend y base de datos

Procedimiento propuesto para 7.6 Implementación del backend y base de datos7.6 Implementación del backend y base de datos7.6 Implementación del backend y base de datos: AG-007 asigna TSK-XXX con entradas y RF/RNF; el agente responsable produce artefacto versionado; AG-006 revisa criterios; el humano aprueba o abre REV-XXX. Evidencias reales pendientes de ejecución.

7.7 Integración y control de versiones

Procedimiento propuesto para 7.7 Integración y control de versiones7.7 Integración y control de versiones7.7 Integración y control de versiones: AG-007 asigna TSK-XXX con entradas y RF/RNF; el agente responsable produce artefacto versionado; AG-006 revisa criterios; el

humano aprueba o abre REV-XXX. Evidencias reales pendientes de ejecución.

7.8 Gestión de fallos, cambios y retrabajo

Procedimiento propuesto para 7.8 Gestión de fallos, cambios y retrabajo7.8 Gestión de fallos, cambios y retrabajo7.8 Gestión de fallos, cambios y retrabajo: AG-007 asigna TSK-XXX con entradas y RF/RNF; el agente responsable produce artefacto versionado; AG-006 revisa criterios; el humano aprueba o abre REV-XXX. Evidencias reales pendientes de ejecución.

7.9 Integración continua, pruebas y despliegue controlado (CI/CD)

Flujo propuesto: rama de tarea → revisión de cambios → análisis estático y dependencias → pruebas unitarias → pruebas de integración/API → pruebas E2E y de autorización → compilación móvil/web → aprobación humana → despliegue a entorno autorizado → verificación posterior.

Política: ninguna incidencia crítica de seguridad podrá quedar abierta para liberar. El pipeline registrará ejecución, versión, responsable y evidencias. No se permitirán despliegues autónomos de agentes a producción ni uso de credenciales productivas en pruebas.

Plan de reversión: conservar artefacto estable anterior, respaldos compatibles y procedimiento documentado de rollback. Definir disparadores de reversión, responsables y pruebas de restauración.

8. CONTROL, SUPERVISIÓN Y SEGURIDAD

8.1 Supervisión humana y aprobaciones

Control propuesto: permisos mínimos, trazabilidad por TSK/REV/CR, revisión de RF/RNF y aprobación humana para cambios críticos. Evidencia: registro de decisiones, revisión y commit o informe cuando se ejecute.

8.2 Límites de acceso de los agentes

Control propuesto: permisos mínimos, trazabilidad por TSK/REV/CR, revisión de RF/RNF y aprobación humana para cambios críticos. Evidencia: registro de decisiones, revisión y commit o informe cuando se ejecute.

8.3 Protección de datos, secretos y credenciales

Control propuesto: permisos mínimos, trazabilidad por TSK/REV/CR, revisión de RF/RNF y aprobación humana para cambios críticos. Evidencia: registro de decisiones, revisión y commit o informe cuando se ejecute.

8.4 Registro y auditoría de cambios

Control propuesto: permisos mínimos, trazabilidad por TSK/REV/CR, revisión de RF/RNF y aprobación humana para cambios críticos. Evidencia: registro de decisiones, revisión y commit o informe cuando se ejecute.

8.5 Gestión de riesgos

Control propuesto: permisos mínimos, trazabilidad por TSK/REV/CR, revisión de RF/RNF y aprobación humana para cambios críticos. Evidencia: registro de decisiones, revisión y commit o informe cuando se ejecute.

8.6 Versionado DERCAS / FigJam / Figma / código

Control propuesto: permisos mínimos, trazabilidad por TSK/REV/CR, revisión de RF/RNF y aprobación humana para cambios críticos. Evidencia: registro de decisiones, revisión y commit o informe cuando se ejecute.

Riesgo	Prevención	Respuesta
Código incorrecto generado	Revisión y pruebas	Revertir cambios
Diseño no alineado con requisitos	Trazabilidad	Corregir prototipo
Acceso no autorizado	Permisos mínimos	Bloquear y auditar
Dependencia de servicios externos	Plan alternativo	Trabajo manual / reintento

Control multiplataforma: una entrega de Figma, frontend o agente de atención no se considerará aprobada si incumple un criterio obligatorio en móvil o escritorio. La incidencia podrá reabrir DERCAS, FigJam, Figma, frontend, backend o pruebas, conservando evidencias por plataforma.

8.7. GESTIÓN DE CICLOS DE REVISIÓN Y CORRECCIÓN ENTRE AGENTES

Objetivo: establecer un proceso iterativo, supervisado y trazable que impida avanzar con entregables incompletos, inconsistentes o inseguros.

8.7.1. Regla principal de la metodología

Cada agente deberá entregar su trabajo acompañado de una verificación contra los criterios establecidos. Si cumple, el coordinador registra su aprobación y habilita la siguiente etapa. Si no cumple, el trabajo se devuelve al agente responsable con observaciones, evidencia y fecha de nueva revisión. Una incidencia podrá reabrir cualquier etapa anterior, no solo programación.

8.7.2. Estados de revisión

Estado	Significado	Siguiente acción
Pendiente	No se ha revisado	Asignar revisor
En revisión	Se están contrastando requisitos y evidencias	Registrar resultado
Observado	Existen incumplimientos	Asignar corrección
En corrección	El agente responsable trabaja en la incidencia	Entregar nueva versión
Aprobado	Cumple los criterios aplicables	Continuar etapa
Bloqueado	Existe un riesgo o dependencia crítica	Escalar a responsable humano

8.7.3. Procedimiento obligatorio

1. El agente recibe una tarea con requisitos, entradas, criterios de aceptación y límites de herramientas.
2. Genera el entregable y lo identifica con código, versión y vínculo de evidencia.
3. El revisor (agente de calidad y/o humano) contrasta el resultado con los criterios.
4. Si es conforme, registra “Aprobado” y autoriza el paso siguiente según el nivel de riesgo.
5. Si es no conforme, abre incidencia con causa, impacto, agente responsable y evidencia.
6. El coordinador determina qué etapas deben reabrirse: DERCAS, FigJam, Figma, desarrollo o pruebas.
7. Se corrige el entregable, se incrementa su versión y se repiten las verificaciones afectadas.
8. Se conserva el historial de decisiones hasta el cierre formal de la incidencia.

8.7.4. Matriz de revisión por etapa

Etapa	Verificación mínima	Si falla, retornar a
DERCAS	Requisitos claros, consistentes y comprobables	AG-001
FigJam / UML	Actores, casos y flujos coinciden con RF	AG-002 o AG-001
Figma / UI	Pantallas y navegación cubren los casos de uso	AG-003, AG-002 o AG-001
Código	Cumple RF, diseño y controles de seguridad	AG-004 / AG-005; reabrir diseño si corresponde
Pruebas	Pasan casos funcionales, integración y seguridad	Etapa que originó la incidencia

8.7.5. Registro de revisión y corrección (plantilla)

ID	Requisito / entregable	Versión	Hallazgo	Responsable	Estado	Evidencia
REV-[ ]	[RF / CU / pantalla / código]	[ ]	[ ]	[ ]	Pendiente	[enlace]
REV-001	RF-065 a RF-068 / seguimiento	v0.1	Ejemplo: acceso a pedido ajeno	AG-004 / AG-005	Observado	[prueba]

8.7.6. Ejemplo completo de retorno entre etapas

Caso: el prototipo Figma de seguimiento de pedidos muestra información de otro cliente. El revisor registra REV-002. El coordinador contrasta RF-065 a RF-068 y el caso de uso de FigJam. Si el requisito es correcto, devuelve el diseño a AG-003; si falta una restricción, reabre DERCAS con AG-001 y actualiza UML con AG-002. AG-004 / AG-005 corrige el código afectado y AG-006 repite pruebas de acceso autorizado y no autorizado. No se declara aprobado hasta comprobar las evidencias.

8.7.7. Condiciones de salida a producción

La publicación solo podrá autorizarse si: (a) los requisitos críticos están cubiertos; (b) no existen incidencias críticas o altas abiertas sin decisión formal; (c) pasan las pruebas funcionales, integración, seguridad y regresión aplicables; (d) se revisan controles de edad, pagos, permisos y privacidad; (e) existe respaldo y plan de reversión; (f) el coordinador humano registra la autorización. Aprobar una prueba no despliega automáticamente el sistema.

8.8 Gobernanza y seguridad de los agentes de IA

Aplicar mínimo privilegio, herramientas en lista permitida, separación de entornos, límites de ejecución y validación humana para borrado, publicación, cambios de permisos, acceso a datos sensibles y pagos. Los agentes no podrán autoelevar privilegios ni alterar sus propias reglas de aprobación.

Riesgos específicos: inyección de instrucciones en documentos y repositorios, fuga de información, resultados inventados, cambios no solicitados y dependencia de servicios externos. Mitigaciones: tratar entradas externas como datos, verificar fuentes, aislar herramientas, auditar llamadas y bloquear operaciones fuera de alcance.

Registro mínimo de auditoría: fecha, agente, tarea, herramienta, acción, recurso, versión, resultado y aprobador; excluir contraseñas, tokens y datos personales innecesarios. Definir

retención y acceso a logs según la política aprobada.

Evento	Control	Evidencia
Lectura de documentos	Fuente y versión autorizada	Referencia y registro
Cambio de código	Rama, alcance y revisión	Commit y PR
Acción crítica	Confirmación humana	Acta o ticket
Intento fuera de permisos	Denegar y alertar	Log sin secretos

8.9 Versionado, reversión y control de cambios integral

Versionar de forma vinculada DERCAS, diagramas, prototipos, código, contratos API y casos de prueba. Cada CR-XXX debe registrar versión origen/destino, justificación, impacto, agente ejecutor, revisor y decisión. Los entregables aprobados no se sobrescriben sin trazabilidad.

Para cada entrega conservar referencia al commit Git y a versiones/enlaces de Figma y FigJam. Si una modificación rompe un criterio, AG-007 abre incidencia, identifica artefactos afectados y ordena la corrección o reversión, con nueva ejecución de pruebas.

9. CRITERIOS DE ACEPTACIÓN

Código	Requisito	Criterio verificable	Evidencia	Estado
CA-001	RF-001	[Condición de aceptación del login]	[Prueba]	Pendiente
CA-AG-001	RAG-002	[Los casos de uso cubren requisitos aprobados]	[Revisión]	Pendiente
CA-UI-001	RDF-001	[Cada pantalla principal tiene vínculo trazable]	[Matriz]	Pendiente
CA-DEV-001	RAG-004	[Los cambios implementados pasan pruebas acordadas]	[Reporte]	Pendiente

9.1 Criterios de aceptación ampliados de las mejoras

Los siguientes criterios son propuestos; registrar evidencia real y estado al ejecutar las pruebas. No marcar como aprobado sin verificación.

Código	Criterio de aceptación	Prueba
CA-ROL-001	Registro público no puede crear Administrador	CP-ROL-001
CA-ROL-002	Usuario no puede consultar pedidos ajenos ni modificar pedidos globales	CP-ROL-002
CA-ORQ-001	Cada tarea tiene entradas, responsable, versión y decisión	CP-ORQ-001
CA-VAL-001	Requisitos críticos tienen criterio y prueba vinculados	CP-VAL-001
CA-TRZ-001	Cambio de requisito actualiza mapa de impactos	CP-TRZ-001
CA-CICD-001	Fallo de prueba obligatoria bloquea integración	CP-CICD-001
CA-SEG-001	Agente sin permiso no puede ejecutar acción crítica	CP-SEG-001
CA-ACC-001	Flujos principales cumplen criterios accesibles acordados	CP-ACC-001
CA-MON-001	Incidencia postpublicación genera ticket y responsable	CP-MON-001

10. PRUEBAS Y VALIDACIÓN

10.1 Pruebas funcionales de la aplicación

Plan de prueba propuesto: preparar datos sintéticos; ejecutar casos positivos y negativos, comparar resultados con RF/RNF, guardar fecha, versión, resultado y evidencia. Estado: pendiente de ejecución.

10.2 Pruebas de integración y seguridad

Plan de prueba propuesto: preparar datos sintéticos; ejecutar casos positivos y negativos, comparar resultados con RF/RNF, guardar fecha, versión, resultado y evidencia. Estado: pendiente de ejecución.

10.3 Pruebas de usabilidad de prototipos

Plan de prueba propuesto: preparar datos sintéticos; ejecutar casos positivos y negativos, comparar resultados con RF/RNF, guardar fecha, versión, resultado y evidencia. Estado: pendiente de ejecución.

10.4 Evaluación del desempeño de los agentes de desarrollo

Plan de prueba propuesto: preparar datos sintéticos; ejecutar casos positivos y negativos, comparar resultados con RF/RNF, guardar fecha, versión, resultado y evidencia. Estado: pendiente de ejecución.

10.5 Verificación de consistencia DERCAS–FigJam–Figma–código

Plan de prueba propuesto: preparar datos sintéticos; ejecutar casos positivos y negativos, comparar resultados con RF/RNF, guardar fecha, versión, resultado y evidencia. Estado: pendiente de ejecución.

Código	Objeto	Procedimiento	Resultado esperado	Estado
CP-001	Login	[Pasos]	[Resultado]	Pendiente
CP-AG-001	Agente de diseño	[Comparar pantalla con requisito]	[Coincidencia]	Pendiente

10.6 Pruebas de compatibilidad móvil y web

CP-MP-001: verificar los flujos principales en Android y, cuando esté disponible, iOS; registrar dispositivo, versión y evidencia.

CP-MP-002: verificar los mismos flujos en Chrome, Edge y Firefox de escritorio; registrar navegador, versión, resolución y evidencia.

CP-MP-003: comprobar navegación responsive, formularios, autenticación, permisos y agentes AAC-001 a AAC-003 en móvil y navegador; cualquier fallo vuelve a la etapa responsable conforme a 8.7.

Criterio de aceptación CA-MP-001: no aprobar la salida a producción hasta superar las pruebas obligatorias de las plataformas incluidas en el alcance, con autorización humana.

10.7 Automatización, regresión y seguridad por roles

Ejecutar CP-ROL-001 (intento de registrarse como Administrador), CP-ROL-002 (Usuario accede a pedido ajeno), CP-ROL-003 (Usuario llama endpoint de inventario), CP-ORQ-001 (tarea sin dependencia aprobada), CP-CICD-001 (pipeline falla), CP-SEG-001 (herramienta prohibida), CP-TRZ-001 (cambio RF), CP-ACC-001 (teclado/contraste), CP-MON-001 (alerta simulada).

Para cada prueba: ID, requisito, entorno, versión, datos sintéticos, pasos, resultado

esperado/obtenido, captura o log, responsable y estado. Ejecutar regresión en módulos afectados por cambios y repetir en Android/iOS y Chrome/Edge/Firefox según el alcance validado.

Prohibido usar pedidos reales o información sensible en pruebas sin base legítima y controles documentados. Las pruebas de seguridad deben ejecutarse solo en sistemas y entornos autorizados.

11. EVALUACIÓN DEL PROYECTO

Indicador	Cómo medirlo	Meta	Resultado
Cobertura de requisitos	Requisitos con pruebas / total	[Meta]	Pendiente
Trazabilidad	Pantallas vinculadas / total	[Meta]	Pendiente
Calidad de código	Pruebas aprobadas / ejecutadas	[Meta]	Pendiente
Calidad de entregables de IA	Entregables aprobados / revisados	[Meta]	Pendiente
Usabilidad	Tareas completadas por usuarios	[Meta]	Pendiente

• Resultados obtenidos: integración documental de propuesta y especificación DERCAS; no se afirma implementación o pruebas realizadas.

• Cumplimiento de objetivos: objetivos de especificación documentados; cumplimiento de desarrollo pendiente de evidencias.

• Limitaciones: faltan enlaces, configuración efectiva de agentes, selección de proveedor IA, cronograma validado y pruebas ejecutadas.

• Mejoras propuestas: completar fichas técnicas por herramienta, enlazar diseños y casos de uso, implementar trazabilidad y registrar pruebas reales.

11.1 Indicadores de calidad, costo y desempeño de agentes

Registrar una línea base antes de fijar metas definitivas. Métricas sugeridas: cobertura RF con CP (%), trazabilidad completa (%), incidencias reabiertas por versión, tasa de aprobación de entregables IA (%), tiempo medio de corrección, fallos de CI/CD, defectos por plataforma, costo estimado por tarea y porcentaje de consultas AAC resueltas con datos verificables.

Frecuencia propuesta: por sprint o hito. AG-007 recopila métricas y un responsable humano interpreta resultados. Un indicador favorable no reemplaza pruebas, aprobación ni verificación de seguridad.

Indicador	**Fórmula o método**	**Meta / resultado**
Cobertura	RF con CP aprobado / RF aplicables × 100	[Definir] / [Medir]
Trazabilidad	RF con vínculos completos / RF aplicables × 100	[Definir] / [Medir]
Reaperturas	Incidencias reabiertas por versión	[Definir] / [Medir]
Costo IA	Costo registrado por tarea o hito	[Definir] / [Medir]
Compatibilidad	Pruebas aprobadas por plataforma / ejecutadas	[Definir] / [Medir]

12. CONCLUSIONES

La integración DERCAS permite organizar requisitos, arquitectura y responsabilidades de

agentes para Licorería Junín Currinchero. Su efectividad deberá evaluarse cuando se ejecuten las tareas y pruebas; no se presume validación técnica.

13. ANEXOS

Anexo A. Requerimientos completos: [Enlace o evidencia]

Anexo B. Diagramas UML de FigJam: [Enlace o evidencia]

Anexo C. Flujos de navegación FigJam: [Enlace o evidencia]

Anexo D. Wireframes y diseños Figma: [Enlace o evidencia]

Anexo E. Prototipo interactivo Figma: [Enlace o evidencia]

Anexo F. Matriz de trazabilidad: [Enlace o evidencia]

Anexo G. Arquitectura de la app: [Enlace o evidencia]

Anexo H. Arquitectura de agentes de desarrollo: [Enlace o evidencia]

Anexo I. Prompts e instrucciones de agentes: [Enlace o evidencia]

Anexo J. Evidencias de código y pruebas: [Enlace o evidencia]

Anexo K. Registro de aprobaciones y cambios: [Enlace o evidencia]

ANEXO F. MATRIZ MAESTRA DE TRAZABILIDAD

Requisito	Caso de uso	Diagrama FigJam	Pantalla Figma	Agente responsable	Código / módulo	Prueba	Estado
RF-001	CU-001	Enlace pendiente	UI-001 Enlace pendiente	AG-004	[Ruta]	CP-001	Pendiente

ANEXOS ADICIONALES PARA LAS 12 MEJORAS

Anexo L. Matriz RBAC y pruebas por rol: [Enlace/evidencia].

Anexo M. Tablero de orquestación, dependencias y fichas TSK: [Enlace/evidencia].

Anexo N. Registro de validación RF y solicitudes CR: [Enlace/evidencia].

Anexo O. Arquitectura de contexto compartido / RAG: [Enlace/evidencia].

Anexo P. Pipeline CI/CD, pruebas de regresión y rollback: [Enlace/evidencia].

Anexo Q. Seguridad de agentes, autorizaciones y auditoría: [Enlace/evidencia].

Anexo R. Indicadores, monitoreo y mantenimiento: [Enlace/evidencia].

CONTROL DE VERSIONES Y APROBACIÓN

Versión	Fecha	Descripción	Responsable	Aprobación
1.1	[Fecha]	Adaptación multiplataforma móvil y web; requisitos y pruebas responsive	[Nombre]	Pendiente
1.2	[Fecha]	Integración de 12 mejoras: RBAC, orquestación, contexto compartido, validación, trazabilidad, CI/CD,	[Responsable]	Pendiente

seguridad, versiones, métricas, RAG, accesibilidad y monitoreo.

NOTA METODOLÓGICA: plantilla adaptada de la estructura DERCAS aportada por el usuario. La orquestación de agentes y la integración Figma/FigJam son extensiones propuestas, no se presentan como una versión oficial de DERCAS.

14. AGENTES DE IA INTEGRADOS EN LA APLICACIÓN MULTIPLATAFORMA

Esta sección complementa los agentes de IA que ayudan a desarrollar el software (sección 2). Los agentes aquí especificados funcionan dentro de la aplicación ya implementada y atienden a sus usuarios. Ambas categorías deben mantenerse diferenciadas en requisitos, arquitectura, permisos y pruebas.

14.1. Objetivo y alcance

Objetivo: [Describir los servicios de atención al cliente que prestarán los agentes dentro de la aplicación multiplataforma].

Canales: chat en Catálogo, Seguimiento y Ayuda/Contacto. Usuarios atendidos: rol Usuario; el Administrador gestiona el soporte humano según permisos.

Canales de ejecución: los agentes AAC-001, AAC-002 y AAC-003 estarán disponibles en la aplicación móvil y en la web de computadoras mediante el mismo backend autorizado; el canal no modifica los permisos del usuario.

14.2. Catálogo de agentes de atención al cliente

Código	Agente dentro de la app	Responsabilidad	Acciones permitidas
AAC-001	Agente de consultas de productos	Responde sobre catálogo, precios y disponibilidad	Consultar catálogo autorizado
AAC-002	Agente de seguimiento de pedidos	Informa el estado de pedidos del cliente	Consultar pedidos propios, previa autenticación
AAC-003	Agente de ayuda y contacto	Responde preguntas frecuentes y deriva casos	Consultar FAQ; crear ticket con consentimiento

Los nombres y funciones anteriores son ejemplos. Añadir, modificar o eliminar agentes según los requisitos reales de la aplicación.

14.3. Ficha de especificación de cada agente

Código y nombre: [AAC-XXX — Nombre]

Propósito y usuario objetivo: AAC-001 consultas de catálogo; AAC-002 seguimiento de pedidos propios; AAC-003 preguntas frecuentes y derivación humana para Usuario.

Disparador y canal de entrada: interacción de chat en Catálogo, Seguimiento y Ayuda/Contacto de la aplicación.

Datos de entrada y contexto permitido: consultas de producto, identificador de pedido propio validado en backend y preguntas frecuentes aprobadas.

Fuentes de conocimiento y herramientas/API: endpoints autorizados de productos, pedidos y

soporte; nunca acceso directo a MySQL.

Reglas de respuesta y formato de salida: respuestas claras en español, sin inventar precios o stock, reconociendo ausencia de información.

Permisos y acciones prohibidas: sin aprobación de transferencias, modificación de estados, compra automática o consulta de pedidos ajenos.

Tratamiento de datos personales y conservación: minimización, autorización por titularidad, no registrar secretos; política de retención pendiente de aprobación.

Escalamiento a persona y manejo de errores: derivar al canal humano, informar fallos de API/proveedor y ofrecer reintentos seguros.

Pantalla Figma y flujo FigJam asociados: [Enlaces]

Requisitos, criterios y pruebas asociados: [Códigos]

Repetir la ficha para cada agente integrado en la aplicación.

14.4. Requerimientos funcionales de atención al cliente

Código	Requerimiento	Prioridad
RF-AC-001	La aplicación permitirá iniciar una conversación con el asistente de atención.	Alta
RF-AC-002	El asistente responderá consultas de productos usando datos autorizados y vigentes.	Alta
RF-AC-003	El asistente permitirá consultar únicamente pedidos asociados al cliente autenticado.	Alta
RF-AC-004	El asistente informará cuando no disponga de datos suficientes o falle una consulta.	Alta
RF-AC-005	El cliente podrá solicitar derivación a atención humana.	Media
RF-AC-006	El sistema mostrará estados de espera, error y respuesta en el chat.	Alta

14.5. Requerimientos no funcionales y específicos de IA

Código	Requerimiento	Verificación
RNF-AC-001	Proteger datos personales y aplicar controles de acceso por usuario.	Pruebas de autorización y privacidad
RNF-AC-002	Registrar eventos necesarios para auditoría sin exponer secretos.	Inspección de registros
RIA-AC-001	No inventar precios, existencias o estados de pedidos.	Casos con datos ausentes
RIA-AC-002	Restringir las herramientas del agente a las expresamente autorizadas.	Pruebas de permisos
RIA-AC-003	Solicitar confirmación y aprobación cuando una acción lo requiera.	Pruebas de acciones sensibles

14.6. Arquitectura de ejecución de agentes dentro de la aplicación

Aplicación móvil y web (Ionic/Angular, Capacitor para móvil) → API backend autenticada → servicio/orquestador de agentes → herramientas de consulta autorizadas → base de datos/servicios. El modelo de IA no debe conectarse directamente a la base de datos con

credenciales privilegiadas. El backend aplica autorización, valida parámetros y registra eventos. [Insertar diagrama FigJam de arquitectura].

Diferencia: los agentes de desarrollo trabajan sobre artefactos DERCAS, Figma, FigJam y repositorios; los agentes de atención operan en producción con permisos limitados y atienden solicitudes del usuario final.

14.7. Caso de uso de ejemplo: consultar estado de pedido

Campo	Descripción
Código	CU-AC-001
Actor	Cliente autenticado
Agente	AAC-002
Precondición	El cliente ha iniciado sesión y tiene un pedido registrado
Flujo principal	1. Cliente abre chat. 2. Solicita estado. 3. Backend verifica identidad y pertenencia. 4. Agente consulta herramienta autorizada. 5. Interfaz muestra estado.
Alternativas	Pedido inexistente, pedido ajeno, API no disponible; informar sin divulgar datos.
Pantalla Figma	[Enlace a Chat / Mis pedidos]
Flujo FigJam	[Enlace al diagrama]

14.8. Diseño de pantallas Figma y flujos FigJam

Código	Artefacto	Contenido mínimo	Enlace
FIG-AC-001	Chat de atención	Campo de texto, enviar, mensajes, carga, error, opción atención humana	Enlace pendiente
FIG-AC-002	Consulta de pedidos	Selector o referencia de pedido y respuesta autorizada	Enlace pendiente
FJ-AC-001	Flujo de conversación	Solicitud → autorización → consulta API → respuesta / error / escalamiento	Enlace pendiente
FJ-AC-002	Diagrama de arquitectura	App → backend → agentes → herramientas → datos	Enlace pendiente

Adjuntar capturas y enlaces de prototipos; las simulaciones de Figma no prueban el funcionamiento del agente.

14.9. Criterios de aceptación y casos de prueba

Criterio	Requisito	Condición verificable	Prueba	Estado
CA-AC-001	RF-AC-001	El usuario abre el chat y envía una consulta.	CP-AC-001	Pendiente
CA-AC-002	RF-AC-003	Solo se muestran pedidos del cliente autenticado.	CP-AC-002	Pendiente
CA-AC-003	RIA-AC-001	Ante datos inexistentes, el agente informa que no puede confirmar el resultado.	CP-AC-003	Pendiente
CA-AC-004	RF-AC-005	El cliente puede solicitar atención humana.	CP-AC-004	Pendiente
CA-AC-005	RF-AC-006	El chat presenta estados de carga y error.	CP-AC-005	Pendiente

14.10. Supervisión, seguridad y riesgos

Riesgo	Control propuesto	Responsable
Divulgación de pedidos ajenos	Autorización en backend por usuario y recurso	[Responsable]
Respuestas inventadas	Consultar fuentes verificadas; comunicar incertidumbre	[Responsable]
Inyección de instrucciones	Tratar entradas y documentos externos como datos no confiables	[Responsable]
Acciones no autorizadas	Lista de herramientas permitidas y confirmación humana	[Responsable]
Falla del servicio IA	Mensaje de error y canal alternativo de atención	[Responsable]

14.11. Trazabilidad integral de ambos tipos de agentes

Requisito app	Agente de desarrollo responsable	Agente dentro de la app	FigJam	Figma	Prueba
RF-AC-003	[Agente desarrollador / QA]	AAC-002	FJ-AC-001	FIG-AC-002	CP-AC-002
RF-AC-006	[Agente UI/UX / frontend]	AAC-001 / AAC-002	FJ-AC-001	FIG-AC-001	CP-AC-005

14.12. Entregables y aprobación

Entregables: [fichas de agentes], [contratos de API], [diagramas FigJam], [prototipos Figma], [código y configuración], [pruebas de seguridad], [evidencias de aceptación], [manual de escalamiento].

Aprobación del responsable funcional: [Nombre / Fecha / Firma]. Aprobación técnica y de seguridad: [Nombre / Fecha / Firma].

Nota: Esta sección es una ampliación propuesta de la plantilla DERCAS; no implica que exista una versión oficial de DERCAS con esta numeración.

14.13 Base de conocimiento, seguridad y mantenimiento de agentes integrados

AAC-001 consultará exclusivamente catálogo, precios y existencias disponibles mediante API de solo lectura. AAC-002 exigirá autenticación y validación de pertenencia del pedido en backend. AAC-003 consultará FAQ aprobadas y derivará a personal humano cuando no pueda resolver la solicitud. Ninguno debe inventar precios, existencias, estados ni confirmar pagos sin verificación.

Si se utiliza RAG para respuestas, la base de conocimiento deberá tener propietario, versión, fecha de actualización, fuentes permitidas y procedimiento de retirada de información obsoleta. No incluir contraseñas, secretos ni pedidos de terceros en el índice.

Mantenimiento: registrar cambios de catálogo y FAQ, medir errores y escalaciones, comprobar disponibilidad de API, revisar periódicamente respuestas con casos de prueba, aplicar cambios versionados y habilitar un canal alternativo cuando falle el servicio IA. Definir responsable humano, frecuencia y evidencia.

Nota legal y funcional: el negocio comercializa bebidas alcohólicas; el sistema debe documentar y validar controles de edad y las restricciones aplicables, sin asumir que el chatbot puede autorizar una venta.

14.14. Fichas operativas propuestas de los agentes de desarrollo

AG-001 — Análisis de requisitos

Objetivo y alcance: Análisis de requisitos para Licorería Junín Currinchero. Entradas: Propuesta y DERCAS. Salidas: Matriz RF/RNF y hallazgos.

Herramientas/plataforma: por asignar cuando el equipo facilite los nombres y conexiones reales; se permiten únicamente herramientas expresamente autorizadas. Restricción: No cambiar requisitos sin CR-XXX.

Dependencias: seguir el ciclo AG-001 → AG-002 → AG-003 → AG-004/AG-005 → AG-006, coordinado por AG-007. Calidad: trazabilidad a RF/RNF, revisión y evidencia. Aprobación: siempre humana. Registro: TSK-XXX, versión, responsable, estado y REV-XXX si procede. Error: bloquear, informar y devolver a la etapa pertinente.

AG-002 — UML y FigJam

Objetivo y alcance: UML y FigJam para Licorería Junín Currinchero. Entradas: RF aprobados y casos de uso. Salidas: Diagramas y especificaciones.

Herramientas/plataforma: por asignar cuando el equipo facilite los nombres y conexiones reales; se permiten únicamente herramientas expresamente autorizadas. Restricción: No inventar enlaces ni afirmar diagramas creados.

Dependencias: seguir el ciclo AG-001 → AG-002 → AG-003 → AG-004/AG-005 → AG-006, coordinado por AG-007. Calidad: trazabilidad a RF/RNF, revisión y evidencia. Aprobación: siempre humana. Registro: TSK-XXX, versión, responsable, estado y REV-XXX si procede. Error: bloquear, informar y devolver a la etapa pertinente.

AG-003 — UI/UX y Figma

Objetivo y alcance: UI/UX y Figma para Licorería Junín Currinchero. Entradas: Casos de uso aprobados. Salidas: Prototipos móvil y escritorio.

Herramientas/plataforma: por asignar cuando el equipo facilite los nombres y conexiones reales; se permiten únicamente herramientas expresamente autorizadas. Restricción: No afirmar accesibilidad sin pruebas.

Dependencias: seguir el ciclo AG-001 → AG-002 → AG-003 → AG-004/AG-005 → AG-006, coordinado por AG-007. Calidad: trazabilidad a RF/RNF, revisión y evidencia. Aprobación: siempre humana. Registro: TSK-XXX, versión, responsable, estado y REV-XXX si procede. Error: bloquear, informar y devolver a la etapa pertinente.

AG-004 — Frontend Ionic/Angular

Objetivo y alcance: Frontend Ionic/Angular para Licorería Junín Currinchero. Entradas: Diseños aprobados y contrato API. Salidas: Componentes, navegación y pruebas frontend.

Herramientas/plataforma: por asignar cuando el equipo facilite los nombres y conexiones reales; se permiten únicamente herramientas expresamente autorizadas. Restricción: No cambiar reglas de negocio ni permisos backend.

Dependencias: seguir el ciclo AG-001 → AG-002 → AG-003 → AG-004/AG-005 → AG-006, coordinado por AG-007. Calidad: trazabilidad a RF/RNF, revisión y evidencia. Aprobación: siempre humana. Registro: TSK-XXX, versión, responsable, estado y REV-XXX si procede. Error: bloquear, informar y devolver a la etapa pertinente.

AG-005 — Backend Node/Express/Sequelize/MySQL

Objetivo y alcance: Backend Node/Express/Sequelize/MySQL para Licorería Junín Currinchero. Entradas: RF/RNF y contratos API. Salidas: Endpoints, servicios, modelos y pruebas.

Herramientas/plataforma: por asignar cuando el equipo facilite los nombres y conexiones reales; se permiten únicamente herramientas expresamente autorizadas. Restricción: No confirmar pagos automáticamente ni exponer datos.

Dependencias: seguir el ciclo AG-001 → AG-002 → AG-003 → AG-004/AG-005 → AG-006, coordinado por AG-007. Calidad: trazabilidad a RF/RNF, revisión y evidencia. Aprobación: siempre humana. Registro: TSK-XXX, versión, responsable, estado y REV-XXX si procede. Error: bloquear, informar y devolver a la etapa pertinente.

AG-006 — Calidad y pruebas

Objetivo y alcance: Calidad y pruebas para Licorería Junín Currinchero. Entradas: RF/RNF, código y diseños. Salidas: Casos, resultados y REV-XXX.

Herramientas/plataforma: por asignar cuando el equipo facilite los nombres y conexiones reales; se permiten únicamente herramientas expresamente autorizadas. Restricción: No declarar aprobadas pruebas no ejecutadas.

Dependencias: seguir el ciclo AG-001 → AG-002 → AG-003 → AG-004/AG-005 → AG-006, coordinado por AG-007. Calidad: trazabilidad a RF/RNF, revisión y evidencia. Aprobación: siempre humana. Registro: TSK-XXX, versión, responsable, estado y REV-XXX si procede. Error: bloquear, informar y devolver a la etapa pertinente.

AG-007 — Orquestación y coordinación

Objetivo y alcance: Orquestación y coordinación para Licorería Junín Currinchero. Entradas: DERCAS, matriz y entregables versionados. Salidas: TSK-XXX, trazabilidad y registro de decisiones.

Herramientas/plataforma: por asignar cuando el equipo facilite los nombres y conexiones reales; se permiten únicamente herramientas expresamente autorizadas. Restricción: No autoaprobar ni desplegar producción.

Dependencias: seguir el ciclo AG-001 → AG-002 → AG-003 → AG-004/AG-005 → AG-006, coordinado por AG-007. Calidad: trazabilidad a RF/RNF, revisión y evidencia. Aprobación: siempre humana. Registro: TSK-XXX, versión, responsable, estado y REV-XXX si procede. Error: bloquear, informar y devolver a la etapa pertinente.